



# 本科毕业设计（论文）

论文题目 一种即插即用的联邦学习特征蒸馏模块设计与实现

作者姓名 朱炳硕

专 业 软件工程

指导教师 郭丁丁教授

2026 年 6 月

# 燕山大学本科毕业设计（论文）

## 一种即插即用的联邦学习特征蒸馏模块设计与实现

学院：人工智能学院  
专业：软件工程  
姓名：朱炳硕  
学号：202211130341  
指导教师：郭丁丁  
答辩日期：□□□□年□月

## 学位论文原创性声明

郑重声明：所呈交的学位论文《一种即插即用的联邦学习特征蒸馏模块设计与实现》，是本人在导师的指导下，独立进行研究取得的成果。除文中已经注明引用的内容外，本论文不包括他人或集体已经发表或撰写过的作品成果。对本文的研究做出贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果，并承诺因本声明而产生的法律结果由本人承担。

学位论文作者签名：

日期： 年 月 日

## 学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权燕山大学将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

保 密 ☐，在\_\_年解密后适用本授权书。

本学位论文属于

不保密 ☐。

(请在以上相应方框内打“√”)

学位论文作者签名：

日期： 年 月 日

指导教师签名：

日期：

年 月 日

## 摘 要

联邦学习能够在原始数据不出本地的条件下协同训练模型，但真实场景中的客户端数据通常呈现显著 Non-IID 分布，导致本地更新方向不一致、全局收敛变慢和模型精度下降。针对该问题，本文设计并实现了一种即插即用的 FedFed 特征蒸馏插件，将特征蒸馏、共享特征池构建和共享特征辅助训练从具体联邦优化流程中解耦，使其能够在不改变主聚合逻辑的条件下接入不同联邦学习基线。本文首先构建多基线联邦学习框架，保留 FedAvg、FedProx、SCAFFOLD、FedAvgM 和 FedNova 等主训练流程，并为 FedFed 插件提供独立辅助信息通道。随后，本文实现基于图像空间特征蒸馏的插件模块：客户端通过生成器和蒸馏分类器学习分类相关敏感特征，服务器按类别维护共享特征池，并在正式联邦训练阶段向客户端下发共享特征，以补充本地数据缺失的类别信息。在 CIFAR-10 强异构场景下，FedFed 插件在 FedAvg 基线上将最高测试准确率由 63.89% 提升至 88.95%；在更强异构设置下，最高准确率由 38.15% 提升至 84.80%；在本地训练轮数增大的场景下，最高准确率由 59.60% 提升至 91.90%。与 FedFed 原方法相比，本文插件化实现牺牲少量主性能，但换取了更清晰的模块边界和跨算法复用能力：在 FedAvg 主实验、更强异构、本地训练增强和 FedProx 扩展设置下，插件结果分别比原方法低 3.39、5.22、1.34 和 1.63 个百分点。跨优化器实验表明，该插件能够接入多种联邦优化器；机制诊断与消融实验表明，特征蒸馏阶段、共享特征参与主训练以及蒸馏增强策略均对性能具有关键作用。隐私实验显示，剪裁与噪声能够降低共享特征的可反演性，但不能稳定抑制成员推断风险。本文工作验证了 FedFed 特征蒸馏思想的插件化可行性，为异构联邦学习中的辅助信息复用提供了一种实现路径。

关键词：联邦学习；数据异构；特征蒸馏；FedFed 插件；Non-IID；隐私验证

## Abstract

Federated learning enables collaborative model training without directly sharing raw data, but non-independent and identically distributed client data often cause client drift, slow convergence, and degraded accuracy. This thesis designs and implements a plug-and-play FedFed feature distillation plugin. The plugin decouples feature distillation, shared feature pool construction, and shared-feature-assisted training from specific federated optimizers, allowing the FedFed mechanism to be reused without changing the main aggregation rule. A multi-baseline framework is built with FedAvg, FedProx, SCAFFOLD, FedAvgM, and FedNova, and an independent auxiliary channel is introduced for the plugin. The plugin learns classification-related sensitive features through image-space distillation, maintains a class-wise shared feature pool on the server, and distributes shared features to clients during formal federated training. On CIFAR-10, the plugin improves FedAvg best accuracy from 63.89% to 88.95% under strong heterogeneity, from 38.15% to 84.80% under stronger heterogeneity, and from 59.60% to 91.90% with more local training. Compared with the original FedFed method, the plugin is 3.39, 5.22, 1.34, and 1.63 percentage points lower in the main FedAvg, stronger heterogeneity, stronger local training, and FedProx extension settings, respectively, while obtaining clearer modular boundaries and cross-algorithm reusability. Mechanism diagnostics, ablation studies, privacy evaluation, and optimizer extension experiments show that feature distillation and shared-feature-assisted training are essential to the plugin, and that clipping with Gaussian noise reduces feature invertibility but does not consistently suppress membership inference risk.

Keywords: federated learning; data heterogeneity; feature distillation; FedFed plugin; privacy evaluation

# 目 录

|                                |    |
|--------------------------------|----|
| 摘 要 .....                      | I  |
| Abstract .....                 | II |
| 第 1 章 绪 论 .....                | 1  |
| 1.1 课题背景概述 .....               | 1  |
| 1.2 现有解决方法 .....               | 1  |
| 1.3 现有方法局限性 .....              | 2  |
| 1.4 本文研究目标与主要内容 .....          | 2  |
| 1.5 本文组织结构 .....               | 3  |
| 第 2 章 相关理论与方法基础 .....          | 4  |
| 2.1 联邦学习基本框架 .....             | 4  |
| 2.2 数据异构与客户端漂移 .....           | 4  |
| 2.3 相关联邦优化算法 .....             | 4  |
| 2.4 特征蒸馏与信息共享 .....            | 5  |
| 2.5 Beta-VAE 与图像空间特征分离 .....   | 5  |
| 2.6 FedFed 方法原理 .....          | 6  |
| 2.7 本章小结 .....                 | 7  |
| 第 3 章 系统总体设计与插件化框架 .....       | 8  |
| 3.1 系统设计目标 .....               | 8  |
| 3.2 系统总体架构 .....               | 8  |
| 3.3 主模型通道与插件通道 .....           | 9  |
| 3.4 插件化接入机制 .....              | 9  |
| 3.5 数据异构场景构造 .....             | 9  |
| 3.6 系统运行流程 .....               | 10 |
| 3.7 本章小结 .....                 | 10 |
| 第 4 章 FedFed 特征蒸馏插件设计与实现 ..... | 11 |
| 4.1 插件模块组成 .....               | 11 |
| 4.2 图像空间特征分离 .....             | 11 |
| 4.3 特征蒸馏目标函数设计 .....           | 12 |

|                          |    |
|--------------------------|----|
| 4.4 两阶段训练机制 .....        | 12 |
| 4.5 共享特征池与辅助训练机制 .....   | 13 |
| 4.6 蒸馏阶段增强策略 .....       | 14 |
| 4.7 隐私扰动与容量控制 .....      | 14 |
| 4.8 本章小结 .....           | 15 |
| 第 5 章 实验设计与性能评估 .....    | 16 |
| 5.1 实验目的 .....           | 16 |
| 5.2 实验环境与基础设置 .....      | 16 |
| 5.3 评价指标与公平性控制 .....     | 16 |
| 5.4 FedFed 插件主性能验证 ..... | 17 |
| 5.5 插件实现路线与配置收敛 .....    | 24 |
| 5.6 插件关键模块消融实验 .....     | 29 |
| 5.7 本章小结 .....           | 33 |
| 第 6 章 机制验证与工程分析 .....    | 34 |
| 6.1 特征蒸馏机制诊断 .....       | 34 |
| 6.2 隐私验证 .....           | 36 |
| 6.3 跨联邦优化器可插拔验证 .....    | 43 |
| 6.4 训练开销分析 .....         | 46 |
| 6.5 本章小结 .....           | 47 |
| 结 论 .....                | 48 |
| 参考文献 .....               | 50 |
| 致 谢 .....                | 52 |





## 第1章 绪论

### 1.1 课题背景概述

随着移动互联网、智能终端和物联网设备的发展，数据大量分布在用户设备、机构系统和边缘节点中。传统集中式机器学习需要汇聚原始数据，容易带来隐私泄露、通信开销和合规风险。联邦学习通过“数据本地保留、模型协同训练”的方式，使多个客户端能够在不直接共享原始数据的条件下共同训练模型，为隐私约束下的分布式建模提供了有效方案[1-3]。

联邦学习通常由服务器和多个客户端组成。服务器初始化全局模型并下发给客户端；客户端基于本地数据训练模型；服务器收集客户端模型参数或更新并完成聚合。FedAvg 是典型联邦优化算法，其通过本地多步训练和服务器加权平均降低通信频率，成为大量联邦学习研究的基础[3]。

真实联邦场景中的客户端数据通常不满足独立同分布假设。不同客户端可能具有不同类别比例、样本数量、采集环境和数据质量。这种 Non-IID 数据分布会导致本地优化方向不一致，引发客户端漂移，使全局模型收敛变慢、精度下降甚至训练不稳定[1,4]。因此，如何在不共享原始数据的前提下缓解数据异构，是联邦学习中的重要问题。

### 1.2 现有解决方法

针对数据异构问题，已有研究形成了多类解决思路。

第一类方法从本地优化目标入手。FedProx 在客户端本地目标中加入近端项，限制本地模型过度偏离全局模型，从而缓解统计异构和系统异构带来的训练不稳定[5]。

第二类方法从更新校正入手。SCAFFOLD 使用服务器端和客户端端控制变量修正本地梯度偏差，降低 Non-IID 数据导致的客户端漂移[6]。

第三类方法从服务器聚合入手。FedNova 通过归一化客户端本地更新，减轻不同本地训练步数导致的目标不一致问题[7]。FedAvgM 则在服务器端引入动量，使全局更新方向更稳定[8]。

第四类方法从模型结构或局部参数保留入手。FedBN 保留客户端本地批归一化

统计量，以处理特征分布偏移问题[9]。

第五类方法从表示约束或共享信息入手。MOON 使用模型级对比学习约束本地表示，减少本地模型偏移[10]。FedProto 通过共享类别原型提高异构客户端之间的语义对齐能力[11]。FedFed 则从图像空间进行特征蒸馏，将原始图像分解为性能敏感特征和性能鲁棒特征，并共享前者以缓解数据异构[12]。

上述方法说明，异构联邦学习已经形成多样化技术路线。不同方法分别修改本地目标、梯度方向、服务器更新、模型结构或共享信息形式。方法多样性提高了性能上限，也使不同机制更容易分散在各自算法流程内。

### 1.3 现有方法局限性

现有异构联邦学习方法通常与具体训练流程结合紧密。优化类方法需要修改本地目标或梯度更新；聚合类方法需要修改服务器更新规则；结构类方法需要改变模型参数共享方式；信息共享类方法需要增加额外数据通道。若每种机制都直接写入某一联邦主流程，系统会出现算法逻辑分散、复用困难和对比成本高的问题。

FedFed 已证明，性能敏感特征共享能够缓解联邦学习中的数据异构，并可通过噪声扰动降低共享特征的隐私风险[12]。本文进一步关注其工程复用形式：将特征蒸馏、共享特征池和辅助训练过程从具体联邦算法中分离，使其能够作为独立模块接入不同联邦基线。

因此，本文不重新设计联邦优化算法，而是实现一种即插即用的 FedFed 特征蒸馏插件。该插件通过独立辅助信息通道完成特征蒸馏和共享训练，在关闭时不影响原始联邦基线，在开启时为基线算法补充跨客户端特征信息。

### 1.4 本文研究目标与主要内容

本文目标是设计并实现一种即插即用的 FedFed 特征蒸馏插件，使其能够接入多种联邦学习基线，并在 Non-IID 图像分类任务中提升模型性能。

本文主要工作如下：

(1) 设计多基线联邦学习框架下的插件化接入机制。系统保留 FedAvg、FedProx、SCAFFOLD、FedAvgM 和 FedNova 等基线的主训练流程，将特征蒸馏相关逻辑放入独立插件通道。

(2) 实现 FedFed 特征蒸馏插件。插件采用 Beta-VAE 风格生成器进行图像空

间特征分离，利用蒸馏分类约束、重构约束和潜变量约束学习性能敏感特征。

(3) 构建共享特征池和辅助训练机制。服务器收集客户端上传的性能敏感特征，按类别维护共享样本池，并在正式联邦训练阶段下发给客户端辅助本地训练。

(4) 完成插件有效性验证。实验包括主性能对比、隐私验证、组件消融、默认配置形成过程、跨联邦基线验证和训练开销分析。

## 1.5 本文组织结构

第 1 章介绍研究背景、现有方法、主要问题和本文工作。第 2 章介绍联邦学习、数据异构、相关联邦优化算法、特征蒸馏、Beta-VAE 和 FedFed 方法基础。第 3 章给出系统总体设计、插件化框架和数据异构构造方式。第 4 章介绍 FedFed 插件的模块组成、训练目标、两阶段流程、共享特征机制和隐私扰动接口。第 5 章围绕实验设计与性能评估展开，验证 FedFed 插件在强异构场景、不同异构强度、本地训练强度、原方法对照、配置收敛和关键模块消融中的表现。第 6 章从特征蒸馏机制、隐私验证、跨联邦优化器接入和训练开销四个角度进行补充分析。最后给出全文结论与后续工作展望。

## 第 2 章 相关理论与方法基础

### 2.1 联邦学习基本框架

联邦学习是一种分布式机器学习范式，其目标是在不集中存储客户端原始数据的条件下训练全局模型[1-3]。

设系统包含  $K$  个客户端，第  $k$  个客户端的数据分布为  $P_k(x,y)$ ，本地目标函数为：

$$L_k(\varphi) = E[(x,y) \sim P_k] \ell(f(x;\varphi), y) \quad (2-1)$$

其中， $\varphi$  为模型参数， $\ell(\cdot)$  为损失函数。全局目标为：

$$\min_{\varphi} L(\varphi) = \sum_{k=1}^K \lambda_k L_k(\varphi) \quad (2-2)$$

其中， $\lambda_k$  为客户端权重，通常由本地样本数决定。每轮训练中，服务器选择部分客户端，下发当前全局模型；客户端完成本地训练后上传更新；服务器聚合更新，得到下一轮全局模型。

### 2.2 数据异构与客户端漂移

联邦学习中的数据异构主要指客户端数据分布不一致。图像分类任务中，常见异构形式包括类别比例偏斜、样本数量偏斜和特征分布偏移[4]。其中，类别比例偏斜会使不同客户端拥有不同标签分布，是联邦图像分类实验中常用的 Non-IID 设置[13]。

数据异构会导致客户端漂移。由于各客户端本地目标不同，本地训练后的更新方向可能偏向各自数据分布。服务器直接聚合这些更新时，全局模型可能偏离总体优化方向。本文采用 Dirichlet 分布构造标签偏斜，并结合数量偏斜模拟异构联邦图像分类场景。

### 2.3 相关联邦优化算法

#### 2.3.1 FedAvg

FedAvg 是联邦学习中的基础算法[3]。客户端接收全局模型后，在本地数据上执行多步随机梯度下降，再将模型参数上传给服务器。服务器按照样本数加权平均：

$$\varphi^{t+1} = \sum_{k \in C_t} [n_k / \sum_{j \in C_t} n_j] \varphi_k^{t+1} \quad (2-3)$$

其中,  $C_t$  为第  $t$  轮参与训练的客户端集合,  $n_k$  为客户端样本数。FedAvg 通信效率高, 但在强 Non-IID 场景下容易受到客户端漂移影响。

### 2.3.2 FedProx

FedProx 在 FedAvg 的本地目标中加入近端正则项, 用于限制本地模型偏离当前全局模型[5]:

$$L_k^{\text{prox}}(\varphi) = L_k(\varphi) + (\mu/2) \|\varphi - \varphi^t\|^2 \quad (2-4)$$

其中,  $\varphi^t$  为当前全局模型参数,  $\mu$  控制近端约束强度。该方法适用于异构程度较高或客户端训练不稳定的场景。

### 2.3.3 SCAFFOLD

SCAFFOLD 使用控制变量校正客户端本地更新方向[6]。服务器和客户端分别维护控制变量, 并利用二者差值修正本地梯度。该方法直接针对客户端漂移问题, 代表了梯度校正类异构联邦优化思路。

### 2.3.4 FedAvgM

FedAvgM 在服务器端引入动量机制, 利用历史全局更新方向平滑服务器更新[8]。相比 FedAvg 直接替换全局模型, FedAvgM 通过服务器动量提升训练稳定性。

### 2.3.5 FedNova

FedNova 针对本地更新步数不一致导致的目标不一致问题, 对客户端更新进行归一化处理[7]。该方法使服务器聚合时显式考虑本地训练强度差异, 从而减轻聚合偏差。

## 2.4 特征蒸馏与信息共享

信息共享可以为客户端补充缺失的全局分布信息。直接共享原始数据会带来隐私风险; 共享 logits、统计量或合成数据也可能泄露敏感信息[12,14,15]。FedFed 的核心思想是共享对模型性能更关键的性能敏感特征, 并将包含更多原始信息的性能鲁棒特征保留在本地[12]。

本文中的特征蒸馏不是传统教师-学生知识蒸馏, 而是指从原始输入中分离出可

用于提升联邦训练性能的图片空间特征。该过程服务于两个目标：一是补充跨客户端类别信息，二是降低直接共享原始数据的风险。

## 2.5 Beta-VAE 与图像空间特征分离

变分自编码器通过编码器、潜变量和解码器学习数据潜在表示[16]。Beta-VAE 在 VAE 基础上增强潜变量正则，使模型在重构能力和表示压缩之间形成更强约束[17]。

FedFed 使用生成器  $q(x)$  建模性能鲁棒特征，并将原始输入与生成器输出之间的差值作为性能敏感特征[12]：

$$x_r = q(x) \quad (2-5)$$

$$x_s = x - x_r \quad (2-6)$$

其中， $x_r$  表示性能鲁棒特征， $x_s$  表示性能敏感特征。本文沿用这一图像空间分离思想，构建 FedFed 插件的特征蒸馏模块。

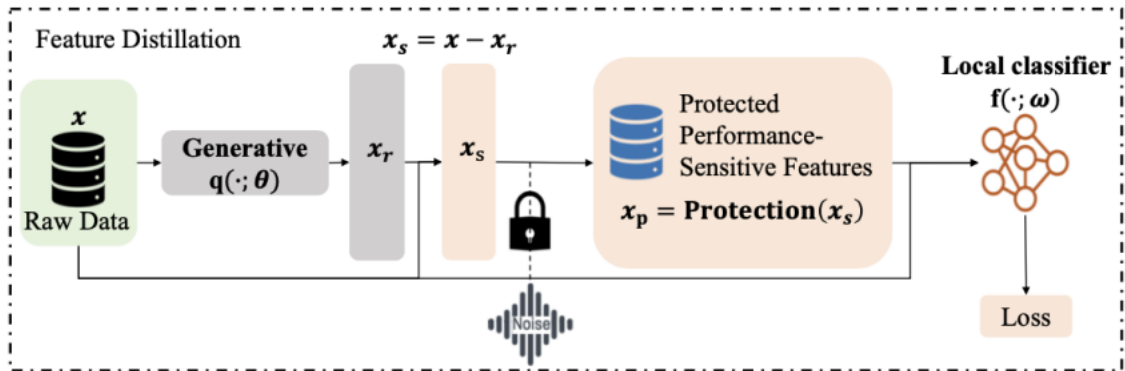


图 2-1 FedFed 方法特征分离机制示意图

图 2-1 展示了 FedFed 的核心思想：生成器输出性能鲁棒特征，原图与鲁棒特征的差值形成性能敏感特征。本文插件的特征提取过程即基于该分离机制。

## 2.6 FedFed 方法原理

FedFed 通过两阶段流程缓解数据异构[12]。第一阶段为特征蒸馏阶段。客户端训练生成器和分类器，使  $x_s = x - q(x)$  保留类别判别信息，并通过范数约束和噪声扰动降低共享风险。第二阶段为正式联邦训练阶段。服务器收集各客户端上传的带扰动敏感特征，构造共享特征集，并下发给客户端辅助本地训练。

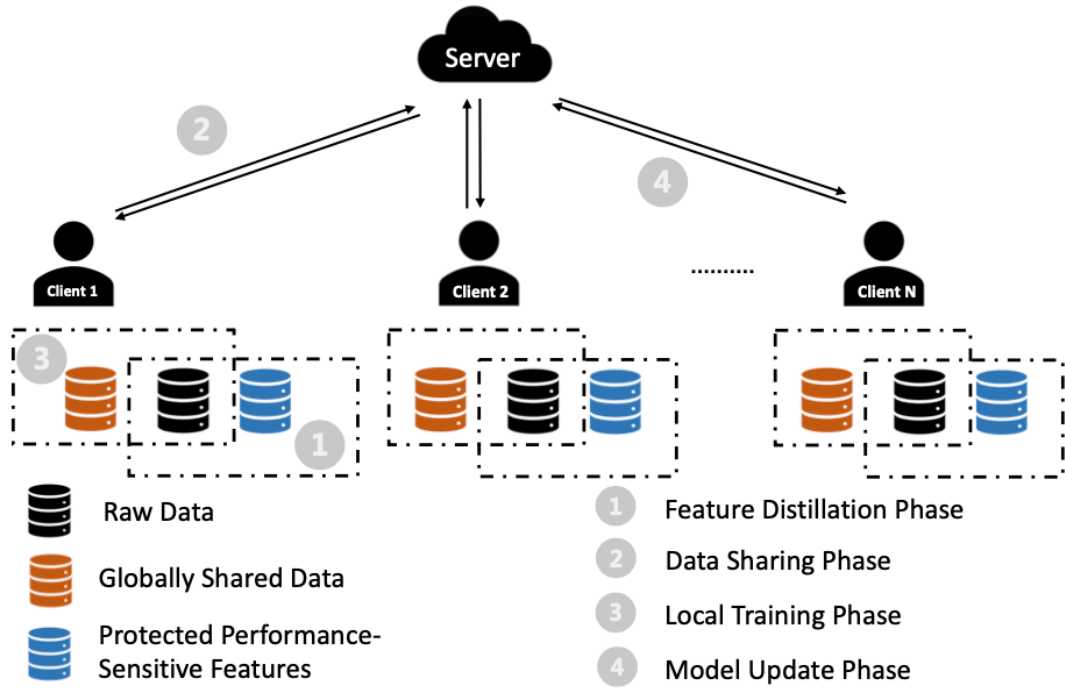


图 2-2 FedFed 工作流程概述示意图

图 2-2 展示了 FedFed 从本地特征蒸馏到共享特征辅助训练的完整流程。该流程是本文插件设计的直接依据。

FedFed 的正式训练目标可写为：

$$\min_{\phi_k} E[(x, y) \sim P_k] \ell(f(x; \phi_k), y) + E[(x_p, y) \sim P_s] \ell(f(x_p; \phi_k), y) \quad (2-7)$$

其中， $x_p$  为加入扰动后的共享敏感特征， $P_s$  为服务器维护的共享特征分布。该目标表明，FedFed 不替代本地训练，而是用共享敏感特征补充跨客户端信息。

## 2.7 本章小结

本章介绍了联邦学习、数据异构、客户端漂移和相关异构联邦优化方法。FedAvg、FedProx、SCAFFOLD、FedAvgM 和 FedNova 分别从参数平均、本地正则、梯度校正、服务器动量和更新归一化角度处理联邦优化问题。FedFed 则从图像空间特征共享角度缓解数据异构。上述内容为后续插件化系统设计和 FedFed 插件实现提供基础。

## 第3章 系统总体设计与插件化框架

### 3.1 系统设计目标

本文系统面向异构联邦图像分类任务，目标是在多种联邦学习基线中接入可启停的 FedFed 特征蒸馏插件。系统设计遵循三个原则。

第一，保留基线主流程。无插件时，系统执行原始 FedAvg、FedProx、SCAFFOLD、FedAvgM 或 FedNova 流程。

第二，解耦特征蒸馏逻辑。特征蒸馏、共享特征收集、共享池维护和辅助训练不写入某一基线算法内部，而通过统一插件通道接入。

第三，保证实验可比性。插件开启与关闭时，数据划分、模型结构、训练轮数和客户端采样规则保持一致，使性能差异主要来自 FedFed 插件。

### 3.2 系统总体架构

系统由三层组成：联邦训练主干层、插件接入层和 FedFed 插件层，如图 3-1 所示。

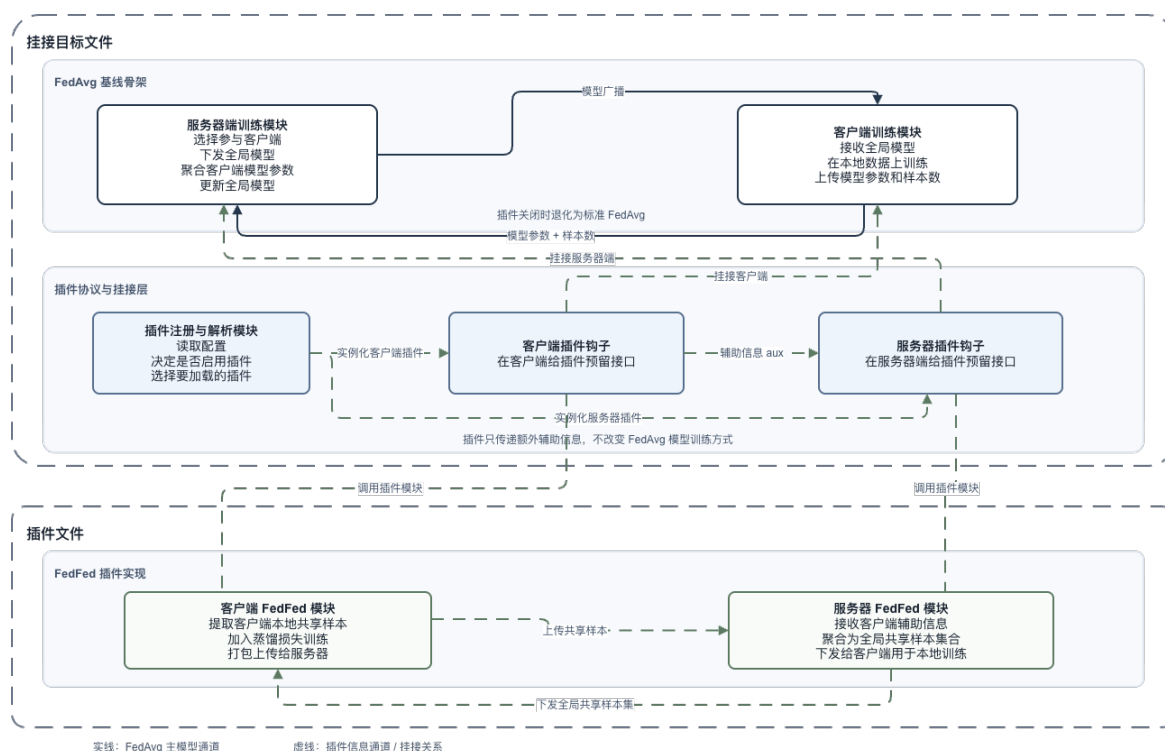


图 3-1 系统总体架构图



联邦训练主干层负责客户端选择、模型广播、本地训练、模型上传、服务器聚合和全局测试。该层承载不同联邦学习基线。

插件接入层负责提供辅助信息通道。该层不改变主模型参数聚合规则，只在客户端训练开始、训练过程、客户端上传和服务器广播等位置传递插件状态。

FedFed 插件层负责图像空间特征蒸馏。客户端侧完成生成器训练、敏感特征提取和辅助信息上传；服务器侧完成插件状态聚合、共享特征池维护和共享特征广播。

图 3-1 的重点是主流程和插件流程分离。主流程保证基线算法可复现，插件流程提供额外特征共享能力。

### 3.3 主模型通道与插件通道

系统将训练信息分为主模型通道和插件通道。主模型通道传递联邦基线所需的信息，包括全局模型参数、本地模型参数、客户端样本数和算法辅助状态。插件通道传递 FedFed 所需的信息，包括生成器状态、蒸馏分类器状态和共享敏感特征。

两条通道职责不同。主模型通道决定基线算法的训练行为；插件通道只为本地训练提供辅助信息，不替代主模型聚合。该设计使 FedFed 插件能够独立启停，并接入不同联邦基线。

### 3.4 插件化接入机制

插件化接入机制围绕客户端侧和服务器侧展开。客户端侧包含三个逻辑环节：轮次开始时接收插件状态和共享特征；本地训练时使用共享敏感特征辅助主模型训练；训练结束后上传插件辅助信息。

服务器侧包含两个逻辑环节：接收客户端辅助信息后更新插件全局状态；下一轮训练前构造插件广播内容。通过该机制，FedFed 插件能够独立维护自身状态，并与不同联邦基线组合。

论文中不展示代码。插件接口通过流程描述和系统图表达，避免将工程实现写成程序说明。

### 3.5 数据异构场景构造

本文主要采用 Dirichlet 分布构造标签分布偏斜。设数据集共有  $C$  个类别，每个类别的样本按照参数为  $\alpha$  的 Dirichlet 分布分配给  $K$  个客户端。参数  $\alpha$  越小，

客户端类别分布差异越大，Non-IID 程度越强[13]。

除标签偏斜外，系统支持客户端样本数量偏斜。该设置使不同客户端拥有不同数量的训练样本，更接近真实联邦场景。本文实验以 CIFAR-10 图像分类为主，重点考察强标签异构和数量异构下 FedFed 插件的作用。

### 3.6 系统运行流程

系统运行分为无插件训练和启用插件训练两种模式。

无插件模式下，系统只执行选定联邦基线的标准流程。客户端仅使用本地数据训练主模型，服务器仅聚合主模型参数。

启用 FedFed 插件后，系统先执行特征蒸馏阶段。客户端训练生成器和蒸馏分类器，学习性能敏感特征。随后服务器收集客户端上传的敏感特征，构建共享特征池。正式训练阶段中，服务器同时下发全局模型和共享特征，客户端使用本地数据与共享特征共同训练主模型。主模型参数仍由对应联邦基线聚合。

### 3.7 本章小结

本章给出了面向多联邦基线的插件化系统设计。系统将主模型通道与 FedFed 插件通道分离，使特征蒸馏模块能够独立启用、关闭和复用。数据异构构造采用 Dirichlet 标签偏斜和数量偏斜，为后续实验提供统一场景。下一章将介绍 FedFed 插件的具体设计与实现。

## 第4章 FedFed 特征蒸馏插件设计与实现

### 4.1 插件模块组成

FedFed 插件由客户端蒸馏模块、服务器共享模块和正式训练辅助模块组成，如图 4-1 所示。

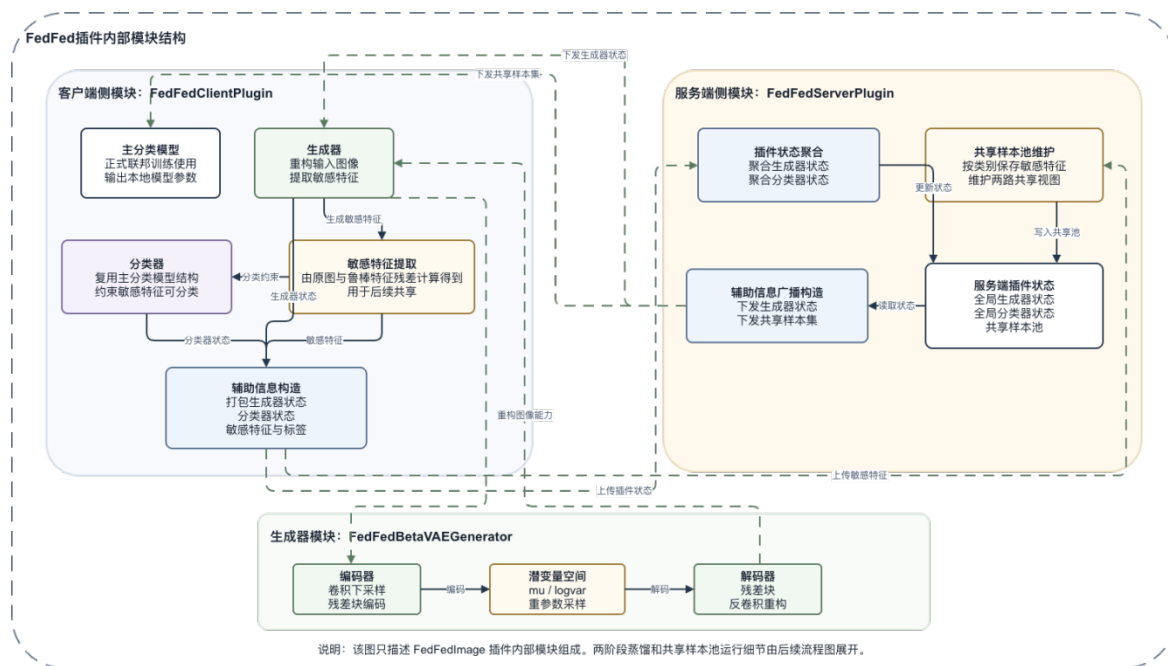


图 4-1 FedFed 插件模块组成图

客户端蒸馏模块负责学习图像空间特征分离。该模块包含生成器和蒸馏分类器。生成器输出性能鲁棒特征，原图与生成器输出之差形成性能敏感特征；蒸馏分类器约束敏感特征具有类别判别能力。

服务器共享模块负责维护插件全局状态。服务器聚合客户端上传的生成器状态和蒸馏分类器状态，并收集客户端上传的敏感特征，形成共享特征池。

正式训练辅助模块负责使用共享敏感特征。客户端训练主模型时，同时利用本地样本和服务端下发的共享样本，从而补充本地数据中缺失的类别信息。

### 4.2 图像空间特征分离

FedFed 插件在图像空间中分离性能鲁棒特征和性能敏感特征。设客户端本地图像为  $x$ ，标签为  $y$ ，生成器为  $q(\cdot; \theta)$ 。生成器输出性能鲁棒特征：

$$x_r = q(x; \theta) \quad (4-1)$$

性能敏感特征由原图与鲁棒特征相减得到：

$$x_s = x - x_r \quad (4-2)$$

其中， $x_r$  主要保留图像主体和冗余信息， $x_s$  保留更有利于分类的判别信息 [12]。本文采用 Beta-VAE 风格生成器实现  $q(\cdot; \theta)$ 。

生成器的作用不是简单复制原图，而是在潜变量约束下学习稳定的鲁棒部分，使差值部分承担分类相关信息。第 5 章中的 NoDistill 消融实验将验证特征蒸馏阶段的作用。

### 4.3 特征蒸馏目标函数设计

特征蒸馏阶段同时优化生成器和蒸馏分类器。设蒸馏分类器为  $g(\cdot; \omega)$ ，交叉熵损失为  $\ell_{ce}$ 。蒸馏目标为：

$$L_{distill} = \lambda_{fd} L_{fd} + \lambda_{rec} L_{rec} + \lambda_x L_x + \lambda_{kl} L_{kl} + \lambda_{logit} L_{logit} \quad (4-3)$$

敏感特征分类损失为：

$$L_{fd} = \ell_{ce}(g(x_s; \omega), y) \quad (4-4)$$

该项约束  $x_s$  保留类别判别信息。重构损失为：

$$L_{rec} = \|q(x; \theta) - x\|^2 \quad (4-5)$$

该项保证生成器输出保持稳定图像结构。原图分类损失为：

$$L_x = \ell_{ce}(g(x; \omega), y) \quad (4-6)$$

该项增强蒸馏分类器的基础判别能力。潜变量正则  $L_{kl}$  来自 Beta-VAE 隐空间约束。输出对齐损失  $L_{logit}$  用于减小原图预测分布与敏感特征预测分布之间的偏差。

### 4.4 两阶段训练机制

FedFed 插件采用两阶段训练流程，如图 4-2 所示。

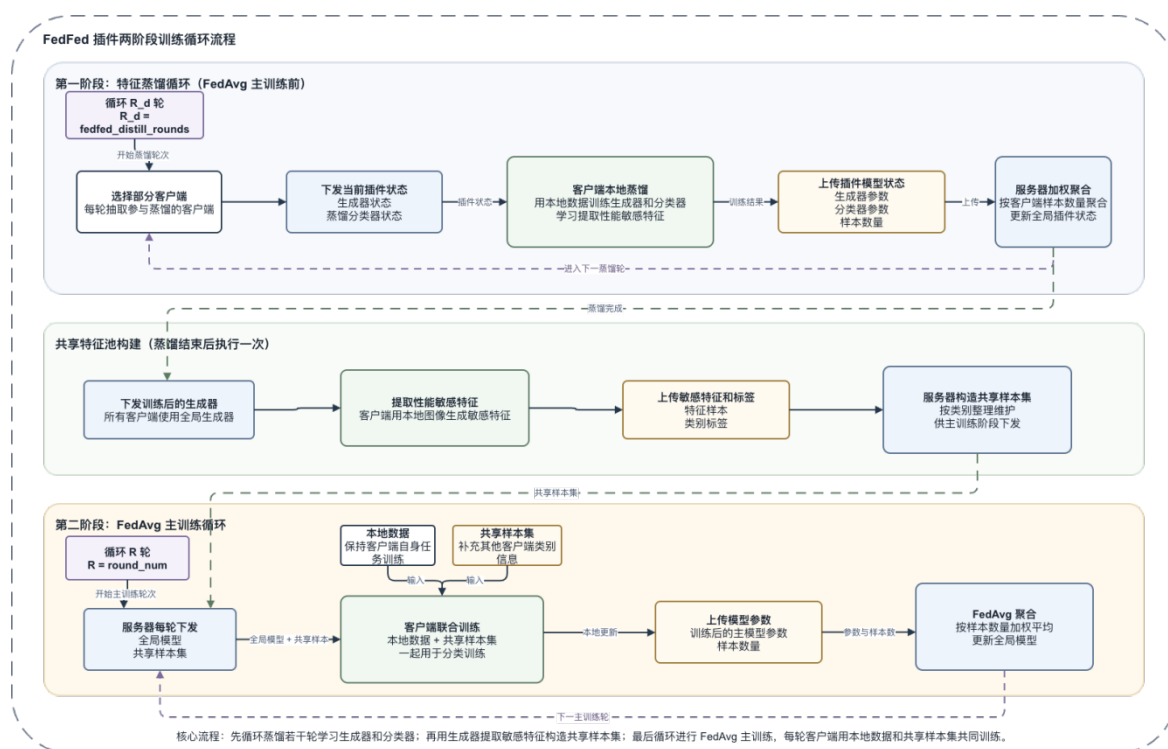


图 4-2 FedFed 插件两阶段训练流程图

第一阶段为特征蒸馏阶段。服务器选择客户端参与蒸馏训练，并下发当前插件状态。客户端在本地数据上训练生成器和蒸馏分类器，使  $x_s$  具有判别能力。训练结束后，客户端上传生成器状态和蒸馏分类器状态，服务器按客户端样本数进行聚合。

蒸馏完成后，系统进入共享特征收集过程。客户端使用聚合后的生成器提取本地性能敏感特征，并上传至服务器。服务器按类别组织这些特征，形成共享特征池。

第二阶段为正式联邦训练阶段。服务器每轮下发全局模型和共享特征。客户端使用本地数据与共享特征共同训练主分类模型。训练结束后，主模型仍按照所选联邦基线进行聚合。

## 4.5 共享特征池与辅助训练机制

共享特征池保存客户端上传的性能敏感特征及其标签，如图 4-3 所示。

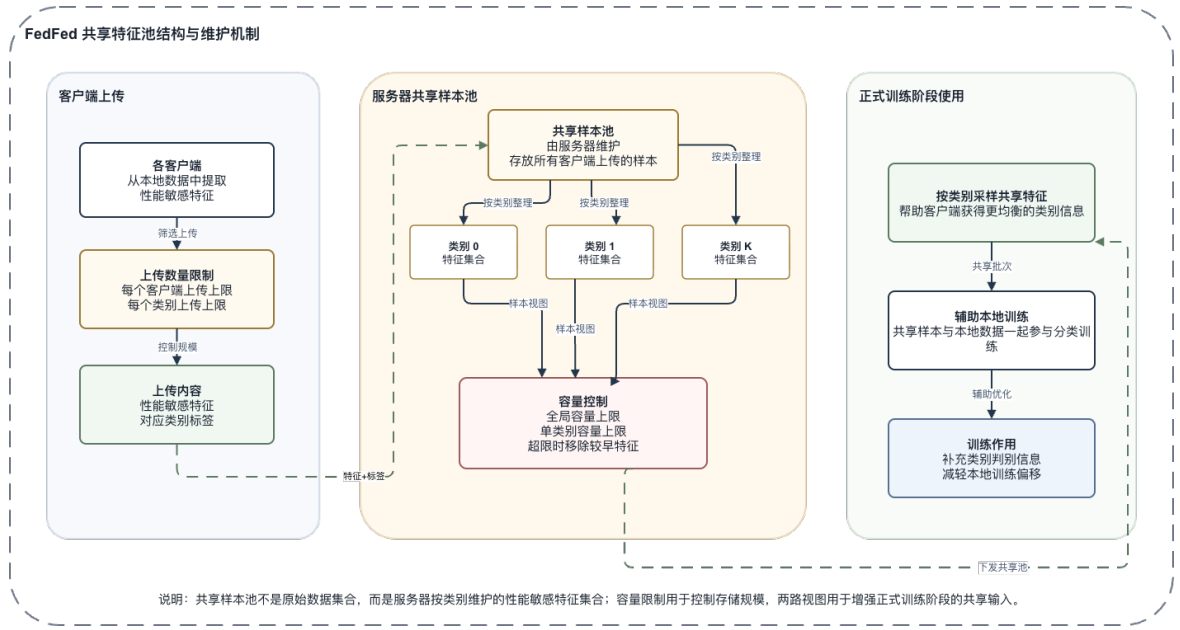


图 4-3 服务器共享特征池结构图

服务器按类别维护共享池，使客户端在正式训练中获得更均衡的类别信息。设主分类模型为  $f(\cdot; \varphi)$ ，本地数据分布为  $P_k$ ，共享特征分布为  $P_s$ 。正式训练阶段的本地目标为：

$$L_{\text{main}} = E[(x, y) \sim P_k] \ell_{\text{ce}}(f(x; \varphi), y) + \lambda_s E[(x_s, y_s) \sim P_s] \ell_{\text{ce}}(f(x_s; \varphi), y_s) \quad (4-7)$$

第一项来自客户端本地数据，第二项来自服务器共享特征。共享特征不替代本地数据，而是补充客户端缺失的全局类别信息。第 5 章中的 NoSharedMainTrain 消融实验将验证该机制的必要性。

## 4.6 蒸馏阶段增强策略

特征蒸馏阶段使用随机裁剪、水平翻转、Mixup 和 Mosaic 等增强策略。随机裁剪和水平翻转增加输入扰动。Mixup 通过样本线性组合增强分类器泛化能力[18]。Mosaic 通过图像拼接提高生成器对复杂输入的适应能力[19]。

这些增强只作用于特征蒸馏阶段，不改变正式联邦训练的服务器聚合规则。第 5 章中的 NoAugMixup 消融实验将验证增强策略对插件稳定性的影响。

## 4.7 隐私扰动与容量控制

FedFed 原论文指出，性能敏感特征仍可能包含样本相关信息，因此需要在共享

前加入噪声扰动[12]。本文在共享前对敏感特征进行范数剪裁，并可加入高斯噪声，形成带扰动共享特征：

$$\mathbf{x}_p = \mathbf{x}_s + \mathbf{n}, \mathbf{n} \sim \mathcal{N}(0, \sigma^2 \mathbf{I}) \quad (4-8)$$

范数剪裁限制单个敏感特征的幅值，噪声扰动降低直接反推原始样本的风险。服务器端同时设置共享池容量上限和单类别容量上限，避免共享池规模失控或类别分布失衡。

需要说明的是，剪裁与高斯噪声属于本文隐私验证配置，不进入第 5 章主性能对比、原方法对照和消融实验。第 6 章将基于带扰动敏感特征分析敏感特征范数、模型反演攻击和成员推断攻击下的隐私表现。

## 4.8 本章小结

本章介绍了 FedFed 插件的设计与实现。插件通过 Beta-VAE 风格生成器完成图像空间特征分离，通过蒸馏目标约束敏感特征的判别能力，并通过两阶段训练构建共享特征池。正式训练阶段中，共享敏感特征作为辅助样本参与本地训练，主模型聚合仍由原联邦基线完成。该设计使 FedFed 能够作为独立插件接入多种联邦学习基线。

## 第 5 章 实验设计与性能评估

### 5.1 实验目的

本章通过 CIFAR-10 数据集验证 FedFed 插件在联邦学习图像分类任务中的性能表现。实验围绕五个问题展开：第一，FedFed 插件是否能够在 FedAvg 基础上提升强异构场景下的分类精度；第二，在不同数据异构程度和本地训练强度下，插件收益是否保持稳定；第三，插件化实现与 FedFed 原方法相比是否能够保留主要性能收益；第四，插件实现路线和配置收敛过程如何影响最终结果；第五，共享特征池规模、蒸馏训练强度和关键结构组件对插件性能的贡献如何。

实验以 FedAvg 作为基础联邦学习算法，将 FedFed 作为可选插件接入同一训练框架。未启用插件时，系统执行标准 FedAvg；启用插件时，系统先进行特征蒸馏与共享特征池构建，再在正式联邦训练阶段利用共享敏感特征辅助本地模型更新。两种方法的服务器聚合规则保持一致，从而保证对比重点集中在插件带来的辅助训练效果上。

### 5.2 实验环境与基础设置

实验数据集采用 CIFAR-10。该数据集包含 10 个图像类别，训练集包含 50000 张图像，测试集包含 10000 张图像。主分类模型采用 ResNet-18。联邦系统设置 10 个客户端，每轮选取 50% 客户端参与训练。正式训练最大通信轮数为 300 轮，本地优化器采用 SGD，学习率为 0.01，动量为 0.9，权重衰减为 0.0001。

数据划分采用 Dirichlet 分布构造标签分布偏斜，并启用客户端样本数量偏斜。Dirichlet 参数  $\alpha$  用于控制数据异构强度， $\alpha$  越小，客户端之间的类别分布差异越强。默认设置为  $\alpha=0.1$ ，对应较强的 Non-IID 场景。

### 5.3 评价指标与公平性控制

本节给出实验评价指标和公平性控制原则。本文主要使用 BestAcc、FinalAcc、BestRound、FinalRound 和 Gain 衡量分类性能。BestAcc 表示训练过程中全局模型在测试集上达到的最高准确率，用于衡量方法的最佳性能；FinalAcc 表示训练结束



或早停时的测试准确率，用于衡量最终稳定性能；BestRound 表示取得最高准确率的通信轮次；FinalRound 表示训练停止时的通信轮次；Gain 表示插件方法相对于无插件基线的 BestAcc 提升。

为保证对比公平，除目标变量外，各组实验保持数据集、数据划分、客户端数量、客户端采样比例、主模型结构、优化器设置、通信轮数和服务器聚合规则一致。无插件基线执行标准联邦训练流程；FedFed 插件方法在相同主流程外额外启用特征蒸馏和共享特征辅助训练。因此，后续主性能实验中的差异主要来自插件是否开启以及插件内部配置变化。

需要说明的是，第 5 章主性能对比、原方法对照、实现路线分析和消融实验均采用未加噪的 FedFed 插件配置。L2 范数剪裁和高斯噪声扰动不进入本章主性能实验，只在第 6 章隐私验证中用于构造带扰动敏感特征并分析特征可用性与隐私风险之间的关系。

## 5.4 FedFed 插件主性能验证

### 5.4.1 主性能对比

主性能对比采用较强数据异构设置。Dirichlet 参数  $\alpha=0.1$ ，本地训练轮数  $E=1$ 。其中， $\alpha$  用于控制客户端标签分布偏斜程度，数值越小表示数据异构越强； $E$  表示每轮通信中客户端在本地数据上训练的 epoch 数。

表 5-1 强异构场景下的主性能对比

| 方法        | $\alpha$ | E | BestAcc | FinalAcc | BestRound | FinalRound | Gain    |
|-----------|----------|---|---------|----------|-----------|------------|---------|
|           |          |   |         |          | d         | d          |         |
| 无插件基线     | 0.1      | 1 | 63.89%  | 53.83%   | 187       | 222        | -       |
| FedFed 插件 | 0.1      | 1 | 88.95%  | 88.53%   | 165       | 167        | +25.06% |

表中， $\alpha$  表示 Dirichlet 标签分布参数， $E$  表示本地训练轮数，BestAcc 表示训练过程最高测试准确率，FinalAcc 表示训练结束时测试准确率，BestRound 表示取得最高准确率的通信轮次，FinalRound 表示实际结束轮次，Gain 表示 FedFed 插件相对于无插件基线的最高准确率提升。由表 5-1 可知，在  $\alpha=0.1$  的强异构场景下，

FedFed 插件将 BestAcc 从 63.89% 提升到 88.95%，将 FinalAcc 从 53.83% 提升到 88.53%，说明开启插件后模型性能和训练稳定性均得到明显改善。

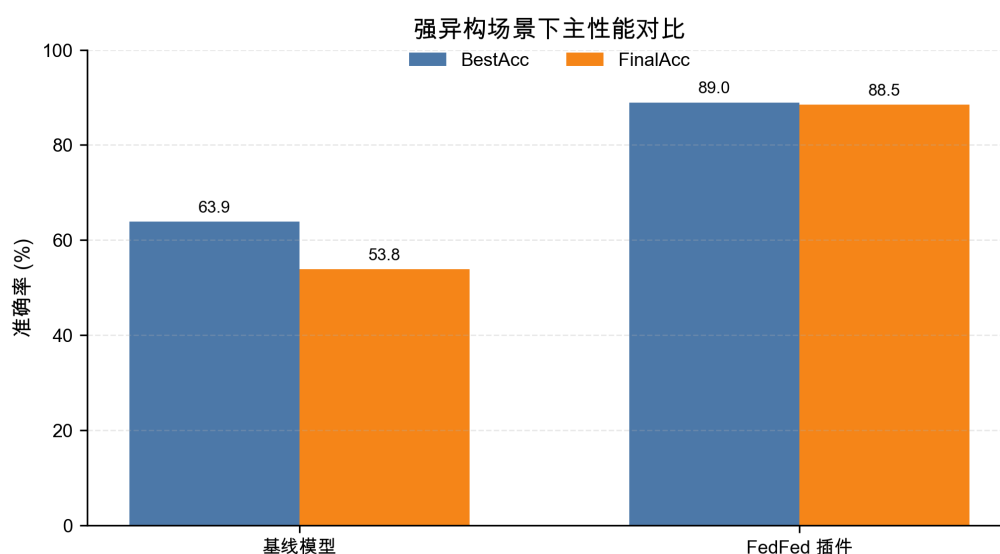


图 5-1 强异构场景下主性能对比

图 5-1 对比了两种方法的最高准确率和最终准确率。无插件基线的最终准确率明显低于最高准确率，说明训练后期存在性能回落；FedFed 插件的 BestAcc 与 FinalAcc 接近，说明插件不仅提升了性能上限，也改善了训练稳定性。

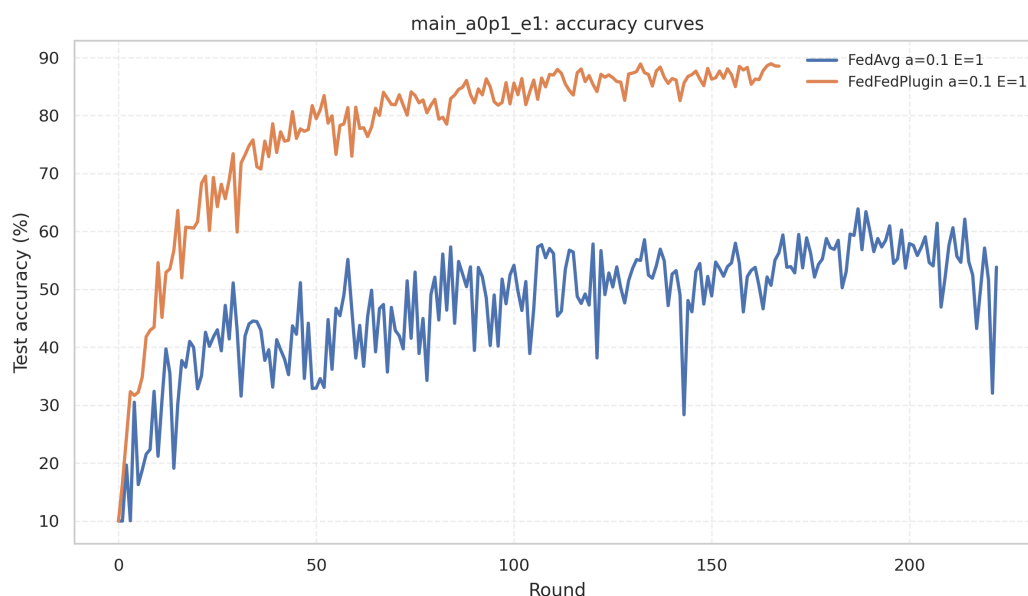


图 5-2 强异构场景下的收敛曲线

图 5-2 展示了训练过程中测试准确率随通信轮次的变化。FedFed 插件在较早轮次达到较高精度，

并在后续训练中保持稳定；无插件基线虽然中途达到一定精度，但后期波动和下降更明显。这说明 FedFed 插件能够缓解 Non-IID 条件下 FedAvg 的收敛不稳定问题。

### 5.4.2 数据异构强度分析

为进一步分析插件收益与数据异构程度之间的关系，本文设置  $\alpha=0.3$ 、 $\alpha=0.1$  和  $\alpha=0.05$  三种数据划分。 $\alpha=0.3$  表示中等异构， $\alpha=0.1$  表示较强异构， $\alpha=0.05$  表示更强的类别分布偏斜。其余训练参数保持一致。

表 5-2 不同数据异构强度下的性能对比

| $\alpha$ | E | 方法        | BestAcc | FinalAcc | Gain    |
|----------|---|-----------|---------|----------|---------|
| 0.3      | 1 | 无插件基线     | 71.95%  | 67.81%   | -       |
| 0.3      | 1 | FedFed 插件 | 91.98%  | 91.14%   | +20.03% |
| 0.1      | 1 | 无插件基线     | 63.89%  | 53.83%   | -       |
| 0.1      | 1 | FedFed 插件 | 88.95%  | 88.53%   | +25.06% |
| 0.05     | 1 | 无插件基线     | 38.15%  | 34.93%   | -       |
| 0.05     | 1 | FedFed 插件 | 84.80%  | 78.64%   | +46.65% |

表中， $\alpha$  表示数据异构强度控制参数，E 表示本地训练轮数，Gain 表示相同  $\alpha$  和 E 下 FedFed 插件相对于无插件基线的 BestAcc 提升。由表 5-2 可知，随着  $\alpha$  变小，无插件基线性能快速下降；相比之下，FedFed 插件在三种异构强度下均保持较高准确率，并在  $\alpha=0.05$  时取得 46.65 个百分点的最大提升。

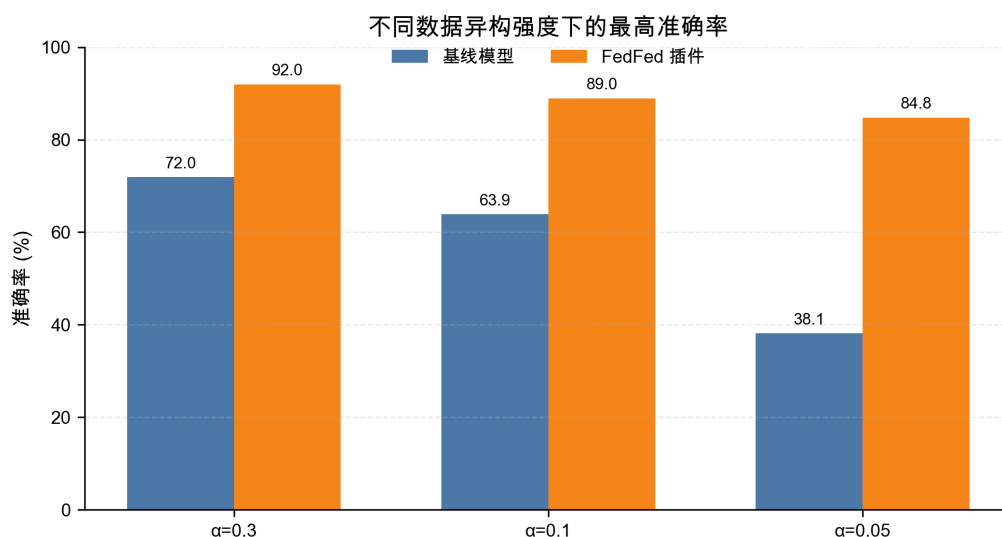


图 5-3 不同数据异构强度下的性能对比

图 5-3 显示，FedFed 插件在不同  $\alpha$  下均明显优于无插件基线。无插件基线对  $\alpha$  的变化更加敏感，而 FedFed 插件的准确率下降幅度较小，说明共享敏感特征能够提高模型对数据异构的鲁棒性。

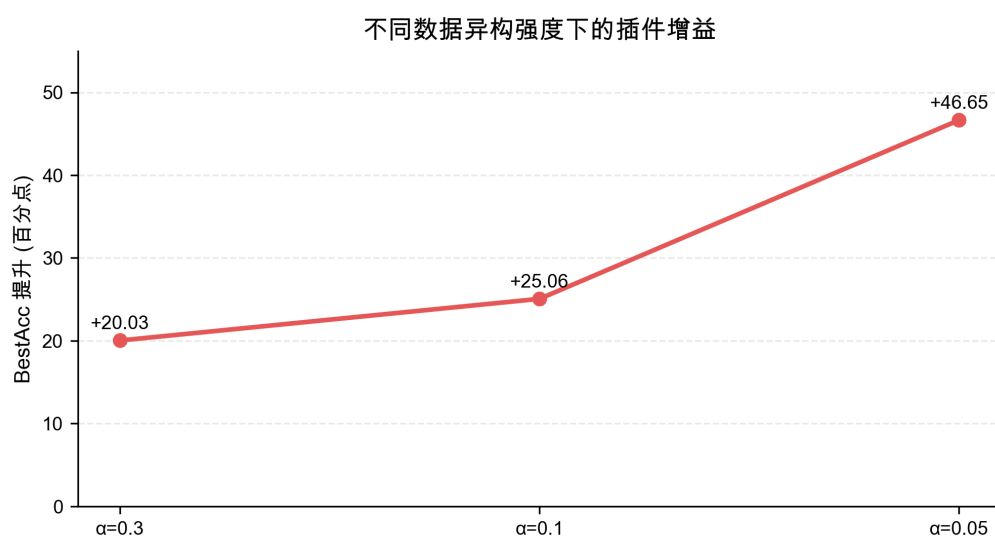


图 5-4 不同数据异构强度下的插件增益

图 5-4 展示了 FedFed 插件在不同异构强度下的准确率增益。随着  $\alpha$  降低，插件增益整体增大，说明客户端数据越偏斜，共享敏感特征提供的跨客户端补充信息越重要。

### 5.4.3 本地训练强度分析

本地训练轮数增大时，客户端会在本地数据上执行更多更新。该设置能够减少通信次数，但在 Non-IID 场景下也可能加剧客户端更新方向差异，形成更明显的 client drift。为验证 FedFed 插件是否能够缓解这一问题，本文将本地训练轮数设置为  $E=5$ ，并与无插件基线进行对比。

表 5-3 本地训练强度增大时的性能对比

| $\alpha$ | E | 方法        | BestAcc | FinalAcc | BestRound | FinalRound | Gain    |
|----------|---|-----------|---------|----------|-----------|------------|---------|
|          |   |           |         |          | d         | d          |         |
| 0.1      | 5 | 无插件基线     | 59.60%  | 58.57%   | 58        | 120        | -       |
| 0.1      | 5 | FedFed 插件 | 91.90%  | 91.01%   | 132       | 167        | +32.30% |

表中，E 表示每轮通信中的本地训练 epoch 数。较大的 E 会增强本地训练强度，也更容易放大 Non-IID 数据下的客户端漂移。由表 5-3 可知，当 E=5 时，无插件基线最高准确率为 59.60%，而 FedFed 插件达到 91.90%，提升 32.30 个百分点，说明插件能够在本地漂移增强时继续保持有效。

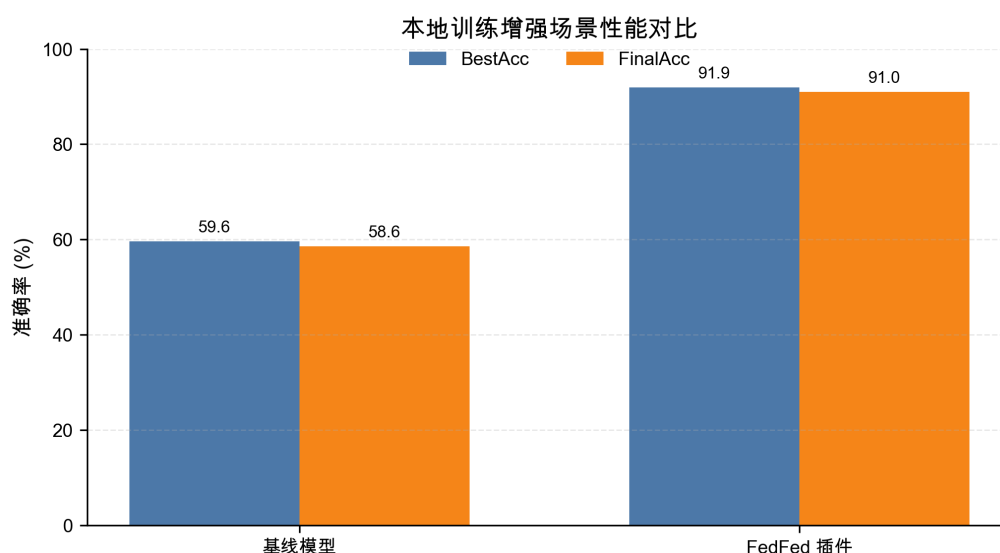


图 5-5 本地训练强度增大时的性能对比

图 5-5 显示，在 E=5 的设置下，FedFed 插件的最高准确率和最终准确率均明显高于无插件基线。这说明共享敏感特征不仅能够补充类别信息，还能在一定程度上约束客户端本地训练方向，使本地更新更接近全局任务目标。

#### 5.4.4 与 FedFed 原方法结果对照

为进一步说明本文插件化实现与 FedFed 方法之间的关系，本文将已有实验结果与 FedFed 原方法在相同任务目的和相近实验设置下进行对照。对照内容包括

FedAvg 主设置、强异构设置、本地训练增强设置，以及 FedProx、SCAFFOLD 和 FedNova 三类扩展优化器设置。

表 5-4 FedFed 原方法与 FedFed 插件结果对照

| 实验目的        | 实验设置                                                  | 基线模型   | FedFed 原方法 | FedFed 插件 |
|-------------|-------------------------------------------------------|--------|------------|-----------|
| 主性能对比       | CIFAR-10,<br>$\alpha=0.1$ , E=1,<br>K=10, FedAvg      | 63.89% | 92.34%     | 88.95%    |
|             | CIFAR-10,<br>$\alpha=0.05$ , E=1,<br>K=10, FedAvg     | 38.15% | 90.02%     | 84.80%    |
| 更强本地训练      | CIFAR-10,<br>$\alpha=0.1$ , E=5,<br>K=10, FedAvg      | 59.60% | 93.24%     | 91.90%    |
| FedProx 扩展  | CIFAR-10,<br>$\alpha=0.1$ , E=1,<br>K=10, FedProx     | 59.91% | 92.12%     | 90.49%    |
| SCAFFOLD 扩展 | CIFAR-10,<br>$\alpha=0.1$ , E=1,<br>K=10,<br>SCAFFOLD | 51.68% | 89.66%     | 56.27%    |
| FedNova 扩展  | CIFAR-10,<br>$\alpha=0.1$ , E=1,<br>K=10, FedNova     | 47.04% | 92.23%     | 70.77%    |

由表 5-4 可知，在 FedAvg 主设置下，FedFed 插件与 FedFed 原方法保持一致趋势。FedFed 原方法在  $\alpha=0.1$ 、E=1、K=10 设置下达到 92.34%，FedFed 插件达到 88.95%，相较基线模型提升 25.06 个百分点。该结果说明，将 FedFed 方法封装为插件后，仍能保留其缓解数据异构的主要效果。

在更强异构设置  $\alpha=0.05$  下，基线模型下降至 38.15%，FedFed 插件仍达到 84.80%。这说明客户端类别分布越偏斜，共享性能敏感特征提供的补充信息越重要。

在  $E=5$  的本地训练设置下, FedFed 插件达到 91.90%, 接近 FedFed 原方法 93.24% 的结果, 说明插件在本地更新更充分、客户端漂移更明显时仍然有效。

从跨优化器结果看, FedProx 和 FedNova 接入 FedFed 插件后均获得明显提升。其中 FedProx 从 59.91% 提升至 90.49%, 与 FedFed 原方法 92.12% 的结果差距较小。FedNova 从 47.04% 提升至 70.77%, 虽然低于 FedFed 原方法结果, 但仍说明插件能够在不同联邦优化器上提供有效补充。

SCAFFOLD 的结果需要单独说明。FedFed 原方法与 SCAFFOLD 直接耦合后达到 89.66%, 而插件化接入后仅达到 56.27%。这说明 SCAFFOLD 的控制变量校正机制与 FedFed 的共享特征训练过程存在更强耦合关系。由于本文插件设计要求不改写底层优化器主体逻辑, 无法像 FedFed 原方法那样深度融合 SCAFFOLD 的更新过程, 因此性能损失较大。但即便如此, FedFed 插件仍高于无插件 SCAFFOLD 的 51.68%, 说明插件在该优化器下仍有一定正向作用。

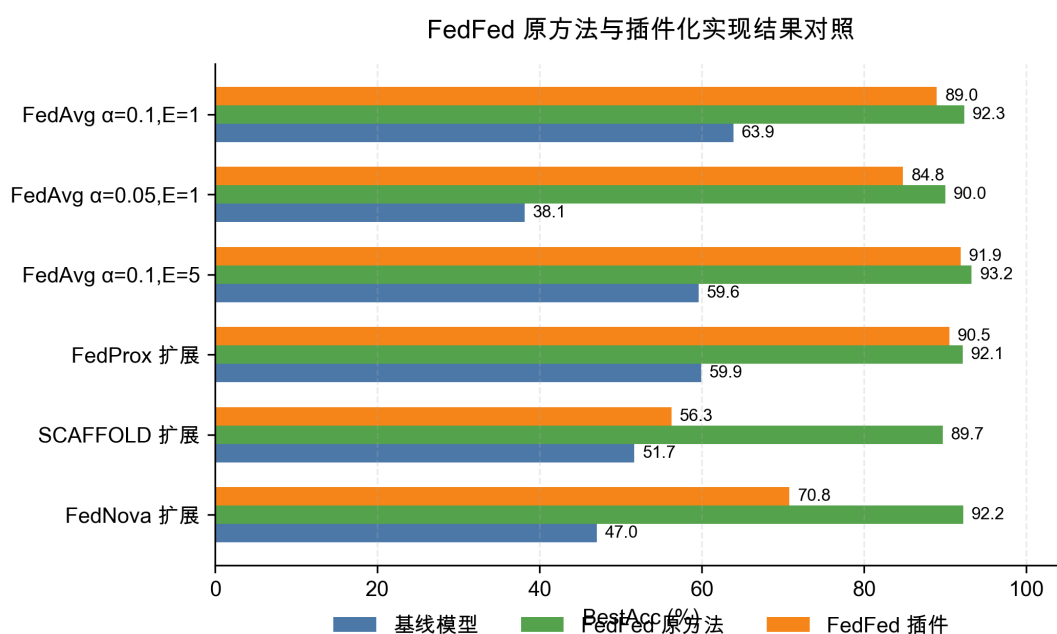


图 5-6 FedFed 原方法与 FedFed 插件结果对照

## 5.5 插件实现路线与配置收敛

### 5.5.1 原型蒸馏插件的早期验证

本文早期首先实现了基于低维特征投影和类别原型共享的 `fedfed_prototype` 插件。该版本通过客户端提取类别原型、服务器聚合共享原型、客户端利用共享原型



进行蒸馏约束的方式接入 FedAvg 主流程。该实现能够验证插件接口的可行性，但实验结果显示其性能收益并不稳定。

在早期 MNIST 强异构实验中，完整 prototype 插件并未稳定超过 FedAvg。部分消融配置甚至优于完整方法，说明低维原型共享和蒸馏约束没有形成可靠的性能来源。随后进行的蒸馏权重扫描显示，较小蒸馏权重能够带来一定改善，但该收益依赖具体参数设置，难以作为最终方法依据。

表 5-5 原型蒸馏插件早期验证结果

| 阶段             | 实现特点      | 代表结果                        | 结论                  |
|----------------|-----------|-----------------------------|---------------------|
| prototype 初版   | 低维投影、类别原型 | FedAvg 84.20% best,         | 完整插件未稳定超过<br>FedAvg |
|                | 共享、蒸馏正则   | Full 82.73% best            |                     |
| prototype 权重调整 | 调整蒸馏损失权重  | $\lambda=0.1$ 时 best 91.89% | 弱正则有一定作用，<br>但机制不稳定 |

上述结果表明，仅共享低维类别原型不足以稳定缓解图像分类任务中的数据异构问题。因此，本文没有将 prototype 插件作为最终方法，而是将其作为插件化接口和早期路线验证。

### 5.5.2 FedFed 插件的确定

基于早期实验结论，本文将实现路线调整为更接近 FedFed 原方法的图像空间特征蒸馏方案，即 FedFed 插件。该版本使用  $\beta$ -VAE 生成器将原始样本划分为性能敏感特征和性能鲁棒特征，并在联邦训练前先进行特征蒸馏阶段，随后将生成的共享特征样本注入 FedAvg 主训练流程。该结构与 FedFed 原方法更加一致，也更适合在 CIFAR-10 等图像任务上验证。

表 5-6 FedFed 插件在不同设置下的性能表现

| 方法        | 数据集      | $\alpha$ | E | best acc | final acc | 说明         |
|-----------|----------|----------|---|----------|-----------|------------|
| 基线模型      | CIFAR-10 | 0.1      | 1 | 63.89%   | 53.83%    | 无插件基线      |
| FedFed 插件 | CIFAR-10 | 0.1      | 1 | 88.95%   | 88.53%    | 插件主结果      |
| 基线模型      | CIFAR-10 | 0.05     | 1 | 38.15%   | 34.93%    | 更强异构       |
| FedFed 插件 | CIFAR-10 | 0.05     | 1 | 84.80%   | 78.64%    | 更强异构下仍保持有效 |
| 基线模型      | CIFAR-10 | 0.1      | 5 | 59.60%   | 58.57%    | 本地训练增强     |
| FedFed 插件 | CIFAR-10 | 0.1      | 5 | 91.90%   | 91.01%    | 缓解本地漂移     |

由表 5-6 可知，FedFed 插件在三类设置下均明显优于基线模型。默认设置下，插件将最高准确率从 63.89% 提升至 88.95%；当  $\alpha$  降低至 0.05 时，基线模型下降明显，而 FedFed 插件仍保持 84.80% 的最高准确率；当本地训练轮数增加至 E=5 时，FedFed 插件达到 91.90%。这些结果说明，FedFed 插件能够同时应对标签分布偏斜和本地训练漂移问题。

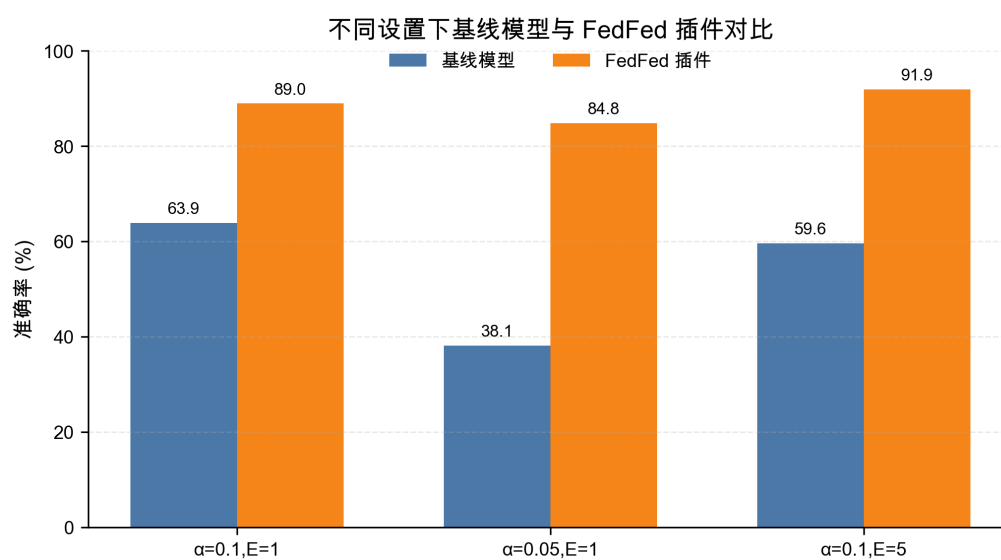


图 5-7 基线模型与 FedFed 插件在不同实验设置下的性能对比

### 5.5.3 实验配置收敛过程

本文实验配置并非一次性确定，而是在多组实验反馈后逐步收敛。早期实验表明，低维原型蒸馏路线性能不稳定；随后采用 FedFed 插件后，CIFAR-10 主实验获得明显提升；进一步的共享池规模实验和蒸馏轮数实验用于确认哪些设置会影响最终性能。

表 5-7 FedFed 插件配置收敛过程

| 实验因素  | 设置          | best acc | final acc | 结论                      |
|-------|-------------|----------|-----------|-------------------------|
| 共享池规模 | SmallPool   | 80.31%   | 75.30%    | 共享特征不足时<br>性能下降         |
| 共享池规模 | MediumPool  | 80.12%   | 75.48%    | 增大到中等规模<br>仍不足          |
| 共享池规模 | FullPool    | 90.33%   | 89.50%    | 充分共享特征时<br>效果最好         |
| 蒸馏轮数  | Distill5    | 89.73%   | 89.43%    | 已能获得较好结<br>果            |
| 蒸馏轮数  | Distill15   | 88.41%   | 87.50%    | 与主配置接近                  |
| 蒸馏轮数  | Distill30   | 89.26%   | 88.73%    | 增加轮数收益有<br>限            |
| 蒸馏轮数  | Distill30E2 | 90.30%   | 88.19%    | 最高值略升，但<br>最终稳定性不更<br>优 |

由表 5-7 可知，共享池规模对插件性能影响明显。SmallPool 和 MediumPool 的最高准确率均约为 80%，明显低于 FullPool 的 90.33%，说明共享特征需要覆盖足够多类别和样本，才能有效补充客户端缺失信息。相比之下，蒸馏轮数的影响较小。Distill5、Distill15 和 Distill30 的最高准确率均处于 88%-90% 区间，说明在特征蒸馏已经充分的情况下，继续增加蒸馏轮数不会带来稳定收益。

因此，本文后续实验采用能够稳定产生有效共享特征的 FedFed 插件配置，并将共享池规模、蒸馏轮数和组件消融作为分析对象，而不是将某一次配置称为固定不变的最终配置。

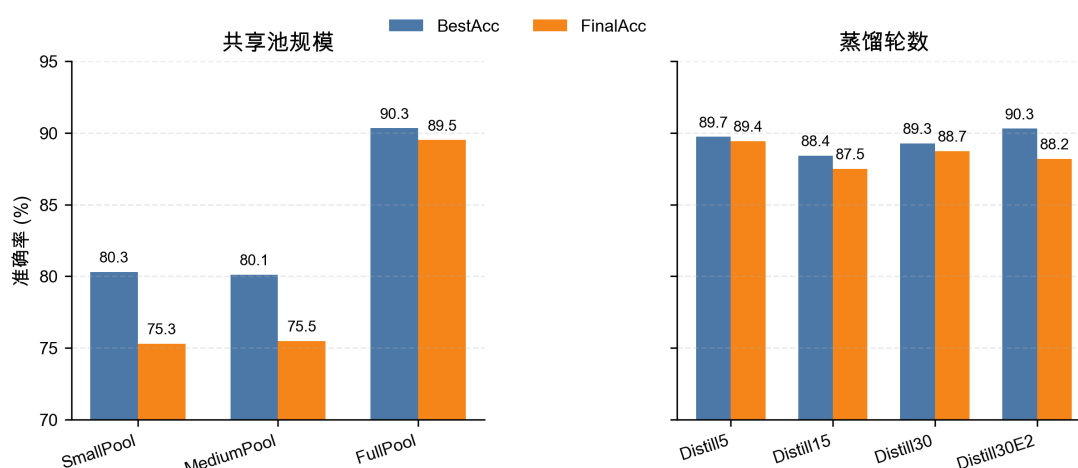


图 5-8 共享池规模与蒸馏轮数对 FedFed 插件性能的影响

## 5.6 插件关键模块消融实验

### 5.6.1 共享特征池规模消融

共享特征池用于保存各客户端上传的性能敏感特征，并在正式联邦训练阶段向客户端提供额外训练样本。为分析共享特征规模对插件性能的影响，本文设置 SmallPool、MediumPool 和 FullPool 三组实验。三组实验保持数据集、模型结构、客户端数量、采样比例和训练轮数一致，仅改变共享特征池容量。

表 5-8 不同共享特征池规模下的性能对比

| 方法         | 上传总量<br>上限 | 单类上传<br>上限 | 池总量上<br>限 | 单类池容<br>量 | best acc | final acc |
|------------|------------|------------|-----------|-----------|----------|-----------|
| SmallPool  | 100        | 4          | 800       | 80        | 80.31%   | 75.30%    |
| MediumPool | 1000       | 20         | 4000      | 400       | 80.12%   | 75.48%    |
| FullPool   | 不限制        | 不限制        | 不限制       | 不限制       | 90.33%   | 89.50%    |

由表 5-8 可知，共享特征池规模对 FedFed 插件性能影响明显。SmallPool 和 MediumPool 的最高准确率分别为 80.31% 和 80.12%，最终准确率均约为 75%。当不对共享池容量设置额外限制时，FullPool 最高准确率达到 90.33%，最终准确率达到 89.50%。该结果说明，在强异构条件下，客户端本地数据存在类别缺失或类别比例偏斜，只有较充分的共享特征池才能为客户端提供有效补充。若共享特征数量过少，插件难以覆盖足够类别信息，性能提升会受到限制。

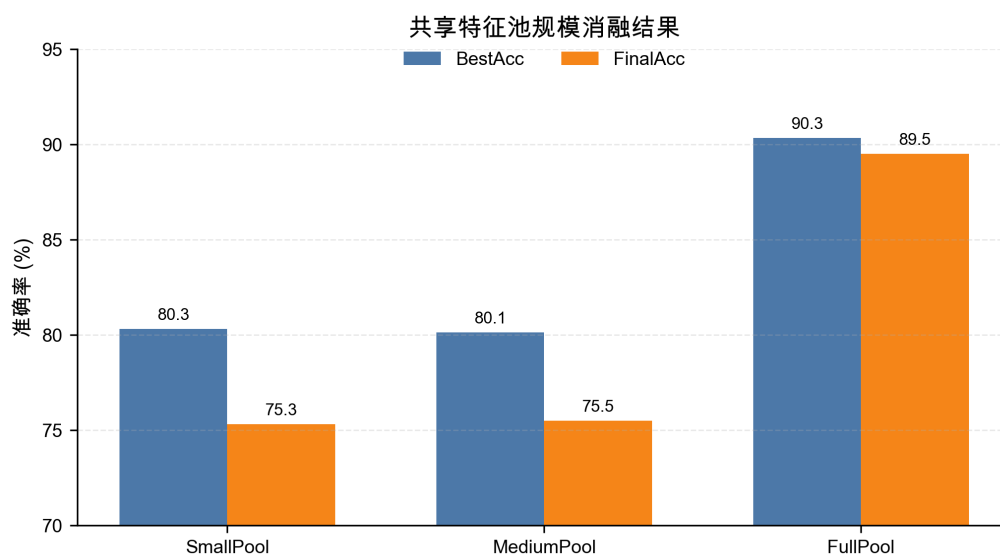


图 5-9 共享特征池规模消融结果

### 5.6.2 蒸馏轮数消融实验

特征蒸馏阶段负责生成可共享的性能敏感特征。为分析蒸馏训练量对插件性能的影响，本文设置 Distill5、Distill15、Distill30 和 Distill30E2 四组实验。其中，Distill5、Distill15 和 Distill30 分别表示特征蒸馏通信轮数为 5、15 和 30；Distill30E2 表示蒸馏通信轮数为 30，且每轮本地蒸馏训练 2 个 epoch。

表 5-9 不同蒸馏轮数下的性能对比

| 方法          | 蒸馏轮数 | 蒸馏本地<br>epoch | best acc | final acc | best round |
|-------------|------|---------------|----------|-----------|------------|
| Distill5    | 5    | 1             | 89.73%   | 89.43%    | 142        |
| Distill15   | 15   | 1             | 88.41%   | 87.50%    | 117        |
| Distill30   | 30   | 1             | 89.26%   | 88.73%    | 157        |
| Distill30E2 | 30   | 2             | 90.30%   | 88.19%    | 212        |

由表 5-9 可知，增加蒸馏轮数并不会持续提升最终性能。Distill5 已经达到 89.73% 的最高准确率和 89.43% 的最终准确率；Distill15 和 Distill30 与其差距较小；Distill30E2 虽然最高准确率达到 90.30%，但最终准确率为 88.19%，稳定性并未进一步提高。该结果说明，特征蒸馏阶段需要达到一定训练量，但在生成特征已经具备分类判别能力后，继续增加蒸馏轮数收益有限。

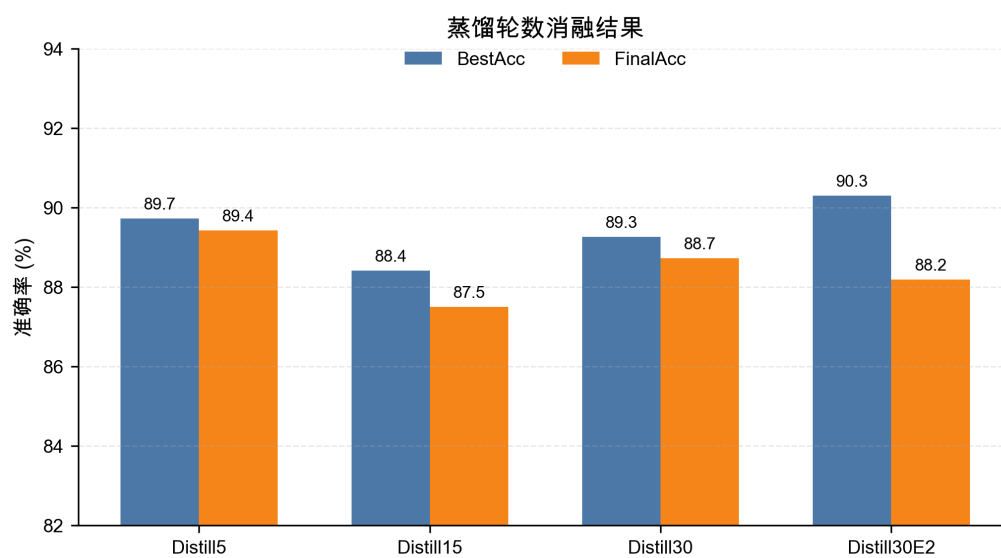


图 5-10 蒸馏轮数消融结果

### 5.6.3 结构组件消融实验

为进一步分析 FedFed 插件的性能来源，本文在 CIFAR-10、 $\alpha=0.1$ 、 $E=1$ 、 $K=10$  设置下进行结构组件消融实验。实验保持数据集、模型结构、客户端数量、采样比例和训练参数一致，仅关闭目标结构，以观察性能变化。

表 5-10 FedFed 插件结构组件消融结果

| 方法                | 关闭内容         | best acc        | final acc       | 结论             |
|-------------------|--------------|-----------------|-----------------|----------------|
| Full              | 完整 FedFed 插件 | 88.95% / 90.33% | 88.53% / 89.50% | 插件完整性性能区间      |
| NoDistill         | 关闭特征蒸馏阶段     | 75.44%          | 74.17%          | 蒸馏阶段是必要条件      |
| NoSharedMainTrain | 共享特征不进入正式训练  | 71.36%          | 63.79%          | 共享特征必须参与主训练    |
| NoAugMixup        | 关闭增强与 mixup  | 76.38%          | 76.16%          | 增强策略显著影响共享特征质量 |

由表 5-10 可知, NoDistill 的最高准确率下降至 75.44%, NoSharedMainTrain 的最高准确率进一步下降至 71.36%。这说明 FedFed 插件的性能提升依赖两个连续环节：首先通过特征蒸馏生成具有分类判别性的共享特征，其次将共享特征用于正式联邦训练。二者缺一不可。

NoAugMixup 的最高准确率为 76.38%，明显低于完整 FedFed 插件。该结果说明，特征蒸馏阶段不仅需要生成共享特征，还需要通过增强策略提高共享特征的多样性和泛化能力。若缺少增强，生成特征更容易局限于本地数据分布，难以充分补充其他客户端缺失的类别信息。

综上，FedFed 插件的核心有效路径可以概括为：通过特征蒸馏构造共享性能敏感特征，再将这些共享特征引入正式联邦训练，从而缓解客户端数据分布偏斜造成的模型更新偏移。消融结果说明，蒸馏阶段、共享特征主训练和增强策略是本文插件实现中最关键的三个组成部分。



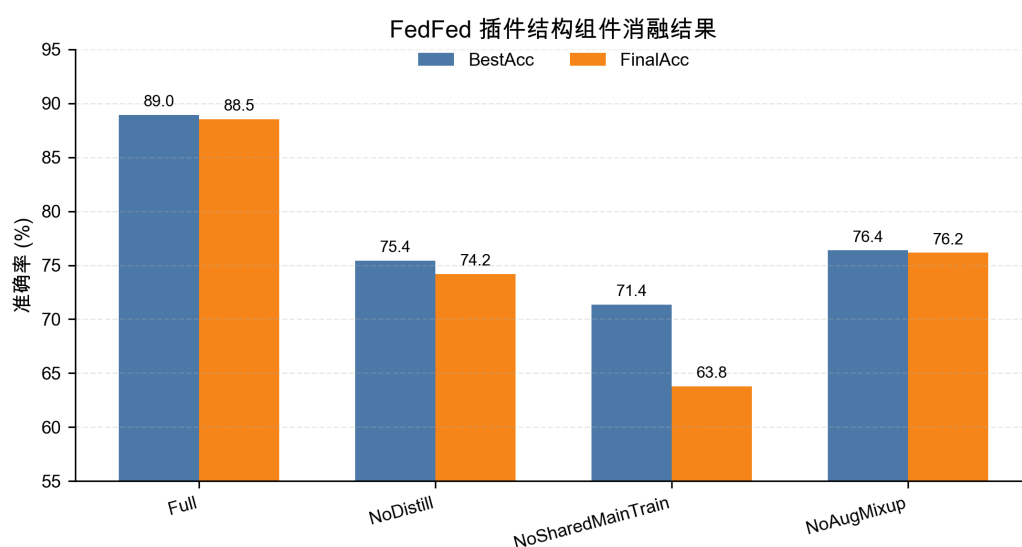


图 5-11 FedFed 插件结构组件消融结果

## 5.7 本章小结

本章围绕 FedFed 插件的性能表现展开实验评估。首先给出实验目的、基础设置和评价指标；随后从主性能、数据异构强度、本地训练强度以及 FedFed 原方法对照四个角度验证插件有效性；最后通过实现路线分析、配置收敛分析和关键模块消融实验说明插件性能来源。实验结果表明，FedFed 插件能够在强异构 CIFAR-10 场景下显著提升基线模型性能，其主要收益来自特征蒸馏、共享特征主训练和增强策略的共同作用。

## 第 6 章 机制验证与工程分析

### 6.1 特征蒸馏机制诊断

为进一步解释 FedFed 插件的性能来源，本文对特征蒸馏阶段得到的不同输入形式进行分类诊断。该实验不直接比较联邦训练最终精度，而是检验原始图像、性能敏感特征、性能鲁棒特征以及带噪敏感特征各自保留的任务信息。

实验设置六类分类代理任务。其中， $x \rightarrow x$  表示使用原始图像训练并在原始图像上测试； $x_s \rightarrow x_s$  表示使用性能敏感特征训练并在性能敏感特征上测试； $x_r \rightarrow x_r$  表示使用性能鲁棒特征训练并在性能鲁棒特征上测试； $x_s \rightarrow x$  表示使用性能敏感特征训练并迁移到原始图像测试； $x + x_s \rightarrow x$  表示同时使用原始图像和敏感特征训练，并在原始图像上测试； $x + x_p \rightarrow x$  表示使用原始图像和带噪敏感特征训练，并在原始图像上测试。

表 6-1 特征蒸馏机制诊断结果

| 代理任务                    | 输入含义          | BestAcc | FinalAcc | BestEpoch |
|-------------------------|---------------|---------|----------|-----------|
| $x \rightarrow x$       | 原始图像分类        | 69.55%  | 69.55%   | 10        |
| $x_s \rightarrow x_s$   | 敏感特征分类        | 74.00%  | 74.00%   | 10        |
| $x_r \rightarrow x_r$   | 鲁棒特征分类        | 54.23%  | 53.53%   | 7         |
| $x_s \rightarrow x$     | 敏感特征到原图迁移     | 14.29%  | 14.29%   | 10        |
| $x + x_s \rightarrow x$ | 原图与敏感特征联合训练   | 73.58%  | 71.29%   | 8         |
| $x + x_p \rightarrow x$ | 原图与带噪敏感特征联合训练 | 71.28%  | 71.22%   | 7         |

表中，BestAcc 表示代理分类器训练过程中的最高测试准确率，FinalAcc 表示训练结束时的测试准确率，BestEpoch 表示取得最高准确率的训练轮次。 $x_s$  表示性能敏感特征， $x_r$  表示性能鲁棒特征， $x_p$  表示加入扰动后的性能敏感特征。

由表 6-1 可知， $x_s \rightarrow x_s$  的最高准确率达到 74.00%，高于原始图像代理任务  $x \rightarrow x$  的 69.55%，说明性能敏感特征中保留了较强的分类判别信息。相比之下， $x_r \rightarrow x_r$  的最高准确率为 54.23%，明显低于  $x_s \rightarrow x_s$ ，说明性能鲁棒特征中的分类信息相对较弱。该结果符合 FedFed 的特征蒸馏目标：敏感特征主要承载与分类任务相关的信息，鲁棒特征更多保留非关键的隐私信息。

同时， $x + x_s \rightarrow x$  的最高准确率达到 73.58%，高于单独使用原始图像的  $x \rightarrow x$ ，说明敏感特征可以作为有效的辅助训练信息。加入扰动后的  $x + x_p \rightarrow x$  仍取得 71.28% 的最高准确率，表明扰动后的敏感特征仍然保留任务可用性。结合前文隐私验证结果可知，带噪敏感特征在保持分类辅助作用的同时，可以降低直接反演原始图像的风险。

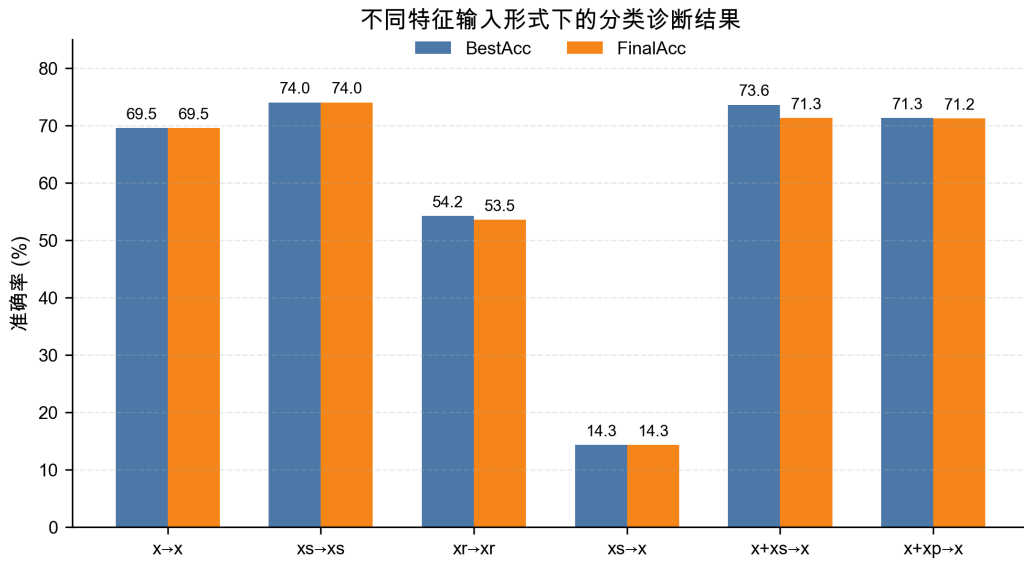


图 6-1 不同特征输入形式下的分类诊断结果

图 6-1 对比了原始图像、敏感特征、鲁棒特征和带噪敏感特征的分类代理性能。结果显示，敏感特征具有较强分类能力，鲁棒特征分类性较弱，说明 FedFed 插件的特征蒸馏过程能够将更多分类相关信息集中到可共享特征中。

## 6.2 隐私验证

FedFed 插件通过共享性能敏感特征缓解客户端数据异构问题，但共享特征本身仍可能带来隐私泄露风险。为验证共享特征的隐私表现，本节在主性能实验之外单独引入 L2 范数剪裁和高斯噪声扰动，并从敏感特征范数、模型反演攻击和成员推断攻击三个角度进行分析。

### 6.2.1 隐私验证思路

本文中特征蒸馏阶段将原始图像  $x$  分解为性能鲁棒特征  $x_r$  和性能敏感特征  $x_s$ ，其中：

$$x_s = x - x_r$$

性能敏感特征  $x_s$  包含较强的分类判别信息，因此可以用于辅助联邦训练。但如果直接共享  $x_s$ ，攻击者可能通过反演攻击恢复原始图像，或通过成员推断攻击判断某个样本是否参与过训练。

因此，本文在隐私验证中对  $x_s$  引入 L2 范数剪裁和高斯噪声扰动。具体做法

为：首先对每个样本的敏感特征进行 L2 范数剪裁，将其范数限制在阈值  $C$  内；随后参考 FedFed 原论文中的噪声注入方式，对剪裁后的敏感特征加入高斯噪声，得到带噪敏感特征  $x_p$ 。该配置仅用于第 6 章隐私验证，不参与第 5 章主性能训练。

本文使用  $x+x_p \rightarrow x$  的分类代理任务评估带噪敏感特征的可用性。这里的  $x+x_p$  并不是指将原始图像和带噪敏感特征逐元素相加，而是指在分类器训练时同时使用原始图像  $x$  和带噪敏感特征  $x_p$  作为训练样本，并在测试时使用原始图像  $x$  进行分类评估。若该分类器仍能取得较高准确率，说明  $x_p$  中仍保留分类任务可用信息，可作为共享特征隐私扰动的可用性代理；该结果不表示主性能实验启用了带噪共享特征。

## 6.2.2 敏感特征剪裁与噪声设置

在确定剪裁与噪声参数前，本文首先统计蒸馏阶段得到的敏感特征  $x_s$  的 L2 范数分布。该实验的目的是观察  $x_s$  的实际数值规模，避免在不了解特征范数的情况下直接设置剪裁阈值或噪声强度。若剪裁阈值过小，会破坏大量有效特征；若阈值过大，则少量异常大范数样本可能削弱固定噪声的扰动效果。

图 6-2 展示了不同配置下敏感特征  $x_s$  的 L2 范数统计结果。

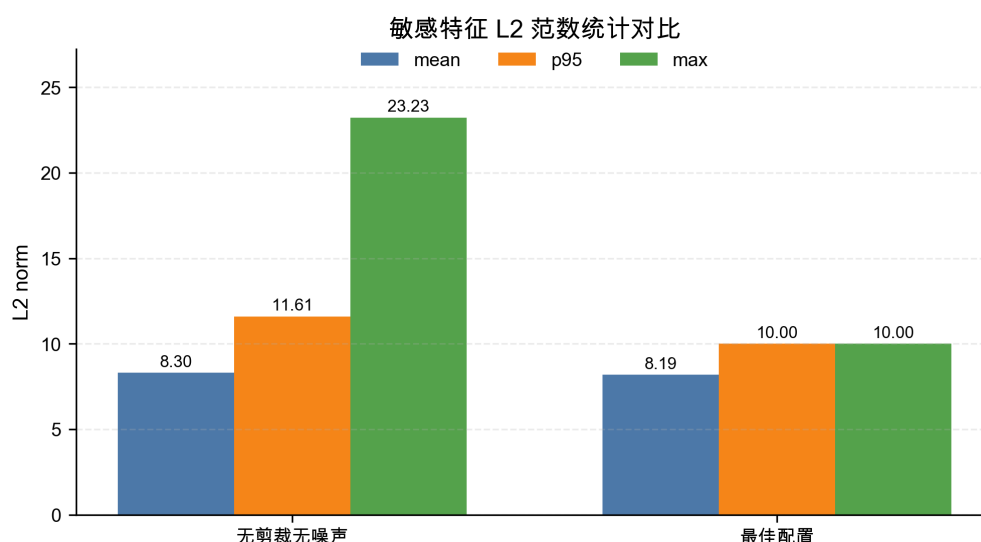


图 6-2 敏感特征 L2 范数统计对比

实验结果显示，在无剪裁无噪声配置下，测试集  $x_s$  的平均范数约为 8.30，p95 为 11.61，p99 为 13.29，最大值达到 23.23。这说明大部分敏感特征集中在 10 左

右，但仍存在少量明显偏大的样本。因此，本文选择  $C=10$  作为剪裁阈值。该阈值能够覆盖主要样本分布，同时限制高范数异常样本。

采用  $C=10$  后，测试集  $x_s$  的  $p95$ 、 $p99$  和最大范数均被限制在 10 附近，说明剪裁机制按预期生效。与此同时，最佳配置下  $x_p$  的平均范数约为 11.68， $p95$  约为 13.09，表明噪声扰动后特征规模仍处于可控范围内，没有出现明显数值失控。

在噪声强度选择上，本文对多组高斯噪声配置进行了对比。实验发现， $C=10$ 、 $\text{Gaussian}(0.15, 0.20)$  在分类代理准确率和反演风险之间取得较好折中：相较无剪裁无噪声配置，分类代理准确率仅下降约 1.22 个百分点，同时反演 PSNR 从 18.87 降至 16.58。因此，本文将该组参数作为后续分析的最佳配置。

最佳配置为：

$$C = 10, \quad \sigma_1 = 0.15, \quad \sigma_2 = 0.20$$

其中， $C$  为敏感特征  $L2$  范数剪裁阈值， $\sigma_1$  和  $\sigma_2$  为两路高斯噪声标准差。本文采用两路噪声设置，主要参考 FedFed 原论文中对敏感特征加入两种强度扰动的思想。其基本考虑是：较弱噪声用于保留敏感特征中的主要判别信息，使带噪特征仍能辅助分类训练；较强噪声用于进一步增强共享阶段的扰动强度，降低敏感特征被直接恢复的风险。通过设置两种噪声强度，系统能够在特征可用性和隐私扰动之间进行折中，而不是只依赖单一噪声强度。

在本文实验中， $\sigma_1$  主要用于构造隐私验证中的带噪敏感特征， $\sigma_2$  用于对应共享阶段更强扰动设置，二者共同构成本文对 FedFed 噪声扰动思想的实现。

### 6.2.3 模型反演攻击验证

模型反演攻击用于评估攻击者能否从共享特征中恢复原始图像。本文训练一个反演网络，以带噪敏感特征  $x_p$  为输入，以原始图像  $x$  为重建目标，并使用 MSE、L1 误差和 PSNR 衡量重建质量。

其中，MSE 表示重建图像与原始图像之间的均方误差，数值越大表示重建误差越大；L1 误差表示像素级绝对误差的平均值，数值越大表示重建结果与原图差异越大；PSNR 表示峰值信噪比，数值越高表示重建图像越接近原图，数值越低表示反演恢复越困难。分类代理准确率  $x+x_p \rightarrow x$  Acc 用于衡量带噪敏感特征对分类任务的可用性，数值越高表示特征仍保留较强任务信息。

表 6-2 模型反演攻击结果

| 配置     | C  | 噪声设置                | $x+x_p \rightarrow x$ Acc | MSE     | L1      | PSNR  |
|--------|----|---------------------|---------------------------|---------|---------|-------|
| 无剪裁无噪声 | -  | None                | 63.33%                    | 0.01296 | 0.08661 | 18.87 |
| 最佳配置   | 10 | Gaussian(0.15,0.20) | 62.10%                    | 0.02196 | 0.11201 | 16.58 |

由表 6-2 可知，最佳配置下分类代理准确率由 63.33% 下降至 62.10%，仅下降约 1.22 个百分点，说明剪裁与噪声对特征可用性的影响较小。同时，模型反演 MSE 由 0.01296 增大至 0.02196，L1 误差由 0.08661 增大至 0.11201，PSNR 由 18.87 降低至 16.58。三项反演指标均表明，攻击者从带噪敏感特征中恢复原始图像的难度明显增加。

为进一步直观观察反演攻击效果，本文分别给出无剪裁无噪声配置和最佳配置下的模型反演可视化结果，如图 6-3 和图 6-4 所示。

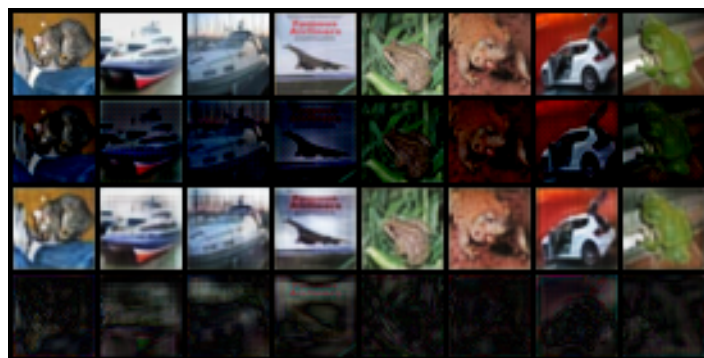


图 6-3 无剪裁无噪声配置下的模型反演结果

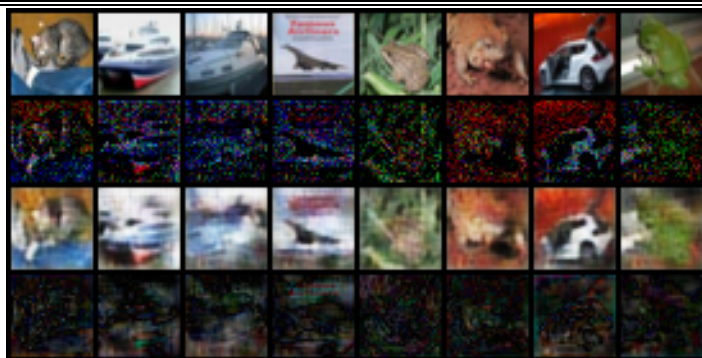


图 6-4 最佳配置下的模型反演结果

图中每组样本依次展示原始图像、共享特征、反演重建图像和重建误差。无剪裁无噪声配置下，反演网络能够从共享敏感特征中恢复出较明显的图像结构和颜色分布；而在最佳配置下，反演重建图像更加模糊，局部细节和类别相关视觉结构明显减弱，重建误差更加突出。该现象与表 6-2 中的定量结果一致，说明剪裁与噪声扰动降低了共享敏感特征的可反演性。

#### 6.2.4 成员推断攻击验证

成员推断攻击用于判断某个样本是否参与过目标模型训练。本文采用基于影子模型的成员推断攻击，即 Shadow Model-based Membership Inference Attack。该攻击假设攻击者无法直接获得目标训练集，但可以获得与目标任务分布相似的辅助数据，并能够查询目标模型输出。在联邦学习场景中，攻击者可以被建模为潜在恶意客户端，其能够获得服务器下发的全局模型，并利用自身持有的同分布数据训练影子模型。

具体流程如下。攻击者首先将辅助数据划分为 shadow train 和 shadow test，其中 shadow train 用于训练影子模型，shadow test 不参与影子模型训练。由于影子模型由攻击者自行训练，因此 shadow train 样本被标记为 member，shadow test 样本被标记为 non-member。随后，攻击者使用与目标模型相同的输入形式训练多个影子模型，并将样本输入影子模型，提取模型输出中的 top-k 概率、置信度、预测间隔、熵、真实类别概率和交叉熵损失等统计特征。最后，攻击者使用这些统计特征训练二分类攻击模型，并将该攻击模型迁移到目标模型上进行成员推断。

为避免攻击模型过弱导致低估成员推断风险，本文进一步增强 Shadow MIA 设置，包括增加 shadow model 数量、增加影子模型训练轮数，并扩大成员推断评估样



本数量。增强后的实验设置和结果如表 6-3 所示。

表 6-3 增强 Shadow MIA 设置下的成员推断攻击结果

| 配置     | Shadow<br>Models | Shadow<br>Epochs | MIA<br>Sample<br>s | MIA<br>AUC | Attack<br>Acc | Member<br>Recall | Member<br>Precision |
|--------|------------------|------------------|--------------------|------------|---------------|------------------|---------------------|
| 无剪裁无噪声 | 5                | 20               | 1500               | 0.702      | 63.43%        | 96.80%           | 58.06%              |
| 最佳配置   | 5                | 20               | 1500               | 0.769      | 69.17%        | 97.73%           | 62.20%              |

其中，Shadow Models 表示训练的影子模型数量；Shadow Epochs 表示每个影子模型的训练轮数；MIA Samples 表示用于目标模型成员推断评估的成员样本和非成员样本数量。MIA AUC 衡量攻击模型区分成员样本和非成员样本的整体能力，越接近 0.5 表示越接近随机猜测，越接近 1 表示攻击能力越强。Attack Acc 表示攻击模型最终二分类准确率；Member Recall 表示真实成员样本中被识别为成员的比例；Member Precision 表示被攻击模型判断为成员的样本中真实成员所占比例。

从增强攻击结果看，无剪裁无噪声配置下 MIA AUC 为 0.702，说明在较强影子模型攻击设置下，攻击者已经能够获得一定成员区分能力。最佳配置下 MIA AUC 进一步升至 0.769，Attack Acc 由 63.43% 提高至 69.17%。该结果表明，剪裁与噪声扰动虽然降低了模型反演风险，但并未稳定降低成员推断风险。

造成这一现象的原因在于，模型反演攻击和成员推断攻击关注的泄露形式不同。模型反演攻击关注能否从共享特征恢复原始图像，而成员推断攻击关注模型对成员样本和非成员样本的输出差异。加入固定噪声后，带噪敏感特征的图像细节被破坏，因此反演难度增加；但固定带噪特征也可能使模型对训练样本产生更明显的输出记忆，从而增大成员样本与非成员样本在置信度、熵和损失等统计特征上的差异。因此，本文认为该剪裁与噪声机制主要降低共享特征的反演风险，而不能视为完整的成员隐私保护方法。

### 6.2.5 隐私验证结果分析

图 6-5 汇总展示了不同配置下的分类代理准确率、MIA AUC 和模型反演 PSNR。

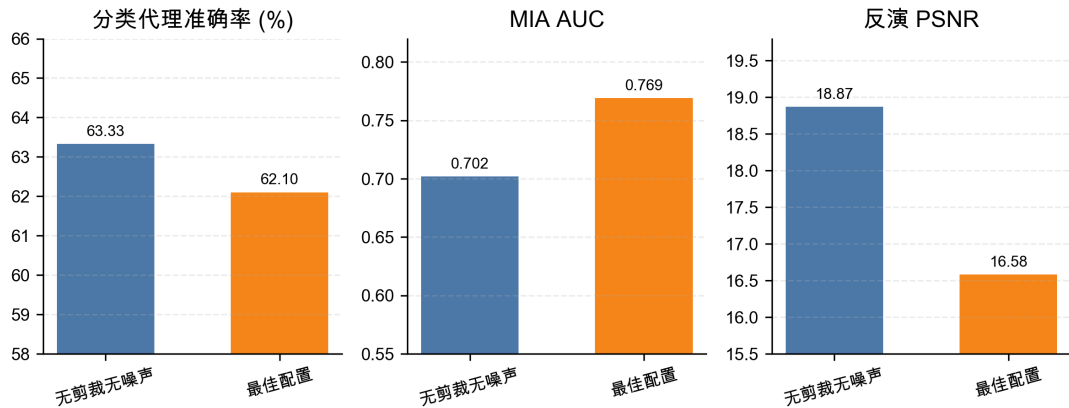


图 6-5 不同配置下的性能与隐私指标对比

由图可知，最佳配置在分类代理准确率上仅有小幅下降，说明其仍保留较好的特征可用性；同时模型反演 PSNR 明显降低，说明其能够削弱共享特征中的可恢复图像信息。但是，MIA AUC 未稳定下降，说明剪裁与噪声并不能完整解决成员推断风险。

综合上述实验，可以得到以下结论。

第一，敏感特征剪裁能够有效限制异常大范数特征。无剪裁时测试集  $x_s$  最大范数达到 23.23，而采用  $C=10$  后， $x_s$  的 p95、p99 和最大范数均被限制在 10 附近，说明剪裁操作能够稳定约束敏感特征规模。

第二，剪裁与高斯噪声能够在较小影响分类代理性能的情况下明显降低模型反演风险。最佳配置下分类代理准确率仅下降约 1.22 个百分点，而反演 PSNR 从 18.87 降至 16.58，说明共享特征中的可恢复图像信息减少。

第三，剪裁与噪声对成员推断攻击的防护效果有限。实验发现，固定带噪敏感特征可能使模型对训练样本和测试样本产生更明显的输出差异，从而导致 MIA AUC 升高。因此，本文不将该机制视为严格的成员隐私保护方法。

综上，本文的隐私增强机制主要用于降低共享敏感特征的可反演性，而不是严格差分隐私机制。实验表明，该方法能够在保持特征可用性的同时降低模型反演风险，但对于成员推断风险仍需结合正则化训练、动态噪声扰动或差分隐私训练等方法进一步改进。该结论与本文工作定位一致，即在 FedFed 特征蒸馏思想基础上进行插件化实现，并对共享特征的隐私风险进行经验性验证。

### 6.3 跨联邦优化器可插拔验证

前述实验主要基于 FedAvg 主流程验证 FedFed 插件的有效性。为进一步验证插件设计的可插拔性,本文将 FedFed 插件接入 FedProx、SCAFFOLD、FedAvgM 和 FedNova 四种联邦优化器,比较开启插件前后的分类性能。该实验用于验证插件接口能否在不同联邦优化主流程中复用,并分析插件效果与底层优化器之间的关系。

表 6-4 不同联邦优化器下的插件接入结果

| 基础优化器    | 插件状态 | BestAcc | FinalAcc | BestRound | FinalRound | Gain    |
|----------|------|---------|----------|-----------|------------|---------|
| FedProx  | 关闭   | 59.91%  | 58.65%   | 133       | 168        | -       |
| FedProx  | 开启   | 90.49%  | 90.25%   | 279       | 300        | +30.58% |
| SCAFFOLD | 关闭   | 51.68%  | 40.33%   | 161       | 196        | -       |
| SCAFFOLD | 开启   | 56.27%  | 53.54%   | 172       | 207        | +4.59%  |
| FedAvgM  | 关闭   | 49.52%  | 43.87%   | 140       | 175        | -       |
| FedAvgM  | 开启   | 80.20%  | 70.86%   | 227       | 262        | +30.68% |
| FedNova  | 关闭   | 47.04%  | 42.73%   | 64        | 120        | -       |
| FedNova  | 开启   | 70.77%  | 62.87%   | 172       | 207        | +23.73% |

表中，基础优化器表示联邦训练使用的主优化方法，插件状态表示是否启用 FedFed 插件，Gain 表示同一基础优化器下开启插件后相对于关闭插件的 BestAcc 提升。

由表 6-4 可知，FedFed 插件能够接入多种联邦优化器，并在 FedProx、FedAvgM 和 FedNova 上带来明显性能提升。其中，FedProx 上最高准确率由 59.91% 提升到 90.49%，FedAvgM 上由 49.52% 提升到 80.20%，FedNova 上由 47.04% 提升到 70.77%。该结果说明，本文插件接口不依赖单一 FedAvg 实现，而是可以作为辅助训练模块接入不同联邦优化流程。

SCAFFOLD 上的提升相对有限，BestAcc 由 51.68% 提升到 56.27%。SCAFFOLD 通过服务器端和客户端控制变量校正本地梯度方向，其核心目标是降低 Non-IID 场景下的 client drift；FedFed 插件则通过共享敏感特征改变客户端本地训练数据分布，为本地分类目标提供额外信息。二者都作用于客户端更新方向，但作用路径不同：SCAFFOLD 从梯度校正角度约束更新，FedFed 从数据补充角度改变本地训练信号。当控制变量估计和共享特征辅助目标同时作用时，客户端更新受到两类校正信号共同影响，插件增益被削弱。因此，跨算法实验表明，FedFed 插件具

备跨联邦优化器接入能力，但性能收益会随底层优化器机制和训练配置变化。

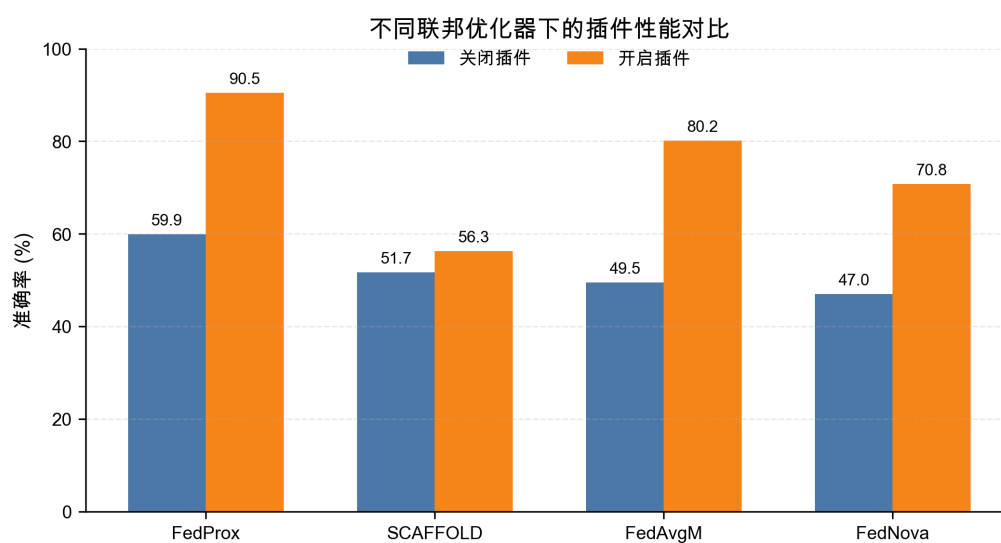


图 6-6 不同联邦优化器下的插件性能对比

图 6-6 展示了不同基础优化器开启和关闭 FedFed 插件后的最高准确率。可以看到，插件在多数优化器上带来性能提升，但提升幅度并不完全一致，说明插件具有可插拔性，同时其效果与底层联邦优化策略存在耦合关系。

## 6.4 训练开销分析

FedFed 插件通过特征蒸馏和共享特征辅助训练提升模型性能，但也会带来额外训练开销。为客观评价插件的工程代价，本文统计不同设置下无插件基线和 FedFed 插件的训练时间，并计算相对耗时倍率。

表 6-5 FedFed 插件训练开销对比

| 实验设置              | 方法        | DistillRound<br>s | 共享池设置  | BestAcc | Duration   | 相对倍率  |
|-------------------|-----------|-------------------|--------|---------|------------|-------|
| $\alpha=0.1, E=1$ | 无插件基线     | 0                 | 无      | 63.89%  | 44.11 min  | 1.00× |
| $\alpha=0.1, E=1$ | FedFed 插件 | 15                | 不设额外限制 | 88.95%  | 163.32 min | 3.70× |
| $\alpha=0.1, E=5$ | 无插件基线     | 0                 | 无      | 59.60%  | 116.03 min | 1.00× |
| $\alpha=0.1, E=5$ | FedFed 插件 | 15                | 不设额外限制 | 91.90%  | 600.50 min | 5.18× |

表中，DistillRounds 表示特征蒸馏阶段的通信轮数，Duration 表示完整实验运行时间，相对倍率表示同一实验设置下 FedFed 插件方法相对于无插件基线的耗时倍数。共享池设置为“不设额外限制”表示实验中未额外限制共享特征池总容量和单类容量。

由表 6-5 可知，在  $\alpha=0.1, E=1$  的主实验设置下，FedFed 插件将最高准确率从 63.89% 提升到 88.95%，训练时间由 44.11 min 增加到 163.32 min，约为无插件基线的 3.70 倍。在  $E=5$  的设置下，FedFed 插件最高准确率达到 91.90%，训练时间增加到 600.50 min，约为无插件基线的 5.18 倍。

该结果说明，FedFed 插件的性能提升伴随额外计算成本。其开销主要来自两部分：一是正式训练前的特征蒸馏阶段；二是正式训练阶段对共享敏感特征的额外训练。当本地训练轮数增大时，共享特征辅助训练的计算量进一步增加。

因此，FedFed 插件适合用于数据异构较强、精度收益需求较高的场景。在实际部署中，需要结合计算资源约束选择蒸馏轮数、共享池规模和本地训练轮数，以在模型性能和训练开销之间取得平衡。

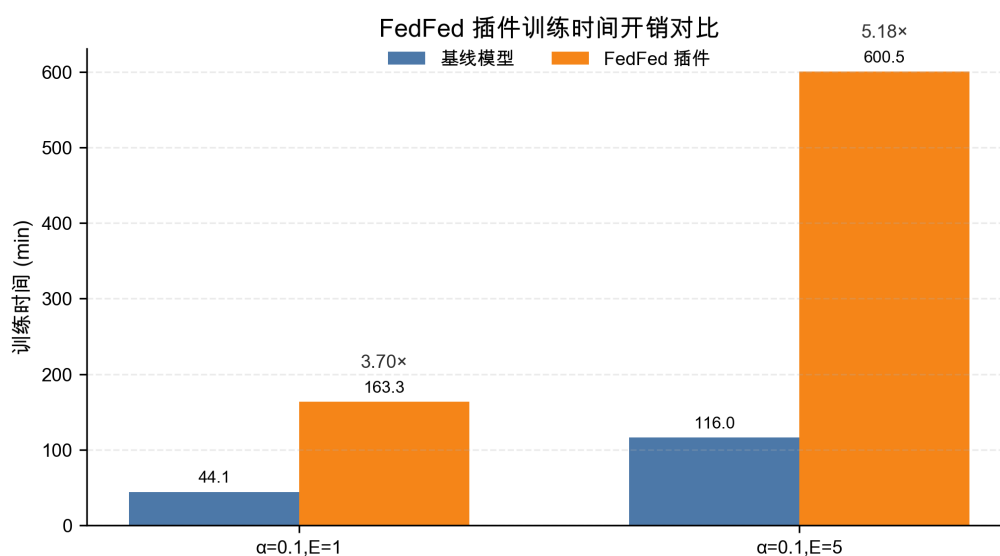


图 6-7 FedFed 插件训练时间开销对比

图 6-7 展示了无插件基线和 FedFed 插件在  $E=1$ 、 $E=5$  两种设置下的训练时间。FedFed 插件带来明显精度提升的同时，也引入额外计算开销，体现了性能收益与工程成本之间的权衡关系。

## 6.5 本章小结

本章从机制、安全性、可插拔性和工程开销四个角度对 FedFed 插件进行补充分析。实验结果表明，插件能够将分类相关信息集中到性能敏感特征中，并具备跨联邦优化器接入能力；同时，隐私验证和开销分析说明该方法仍需要在隐私风险控制和训练成本之间进行权衡。

## 结 论

本文围绕异构联邦学习中的性能退化问题，设计并实现了一种即插即用的 FedFed 特征蒸馏插件。该插件不重新定义联邦优化算法，而是在原有主训练流程之外建立独立辅助通道，将特征蒸馏、共享特征池和共享特征辅助训练封装为可启停模块。关闭插件时，系统执行原始联邦基线；开启插件时，客户端获得跨客户端共享的分类相关特征，从而缓解本地类别分布偏斜带来的训练偏移。

本文首先实现了多基线联邦学习框架下的插件化接入机制。系统保留 FedAvg、FedProx、SCAFFOLD、FedAvgM 和 FedNova 的主流程，将插件状态维护、共享特征收集和辅助训练过程从基线算法中分离，提高了方法复用性和实验可比性。

实验结果表明，FedFed 插件能够在强 Non-IID 场景下显著提升分类性能。在 CIFAR-10、 $\alpha=0.1$ 、 $E=1$  设置下，FedAvg 最高准确率为 63.89%，开启插件后提升至 88.95%；在  $\alpha=0.05$  的更强异构设置下，插件将最高准确率由 38.15% 提升至 84.80%；在  $E=5$  设置下，插件将最高准确率由 59.60% 提升至 91.90%。这些结果说明，共享特征辅助训练能够有效缓解数据异构造成的性能下降。

与 FedFed 原方法相比，本文插件化实现以少量核心性能损失换取了更强的系统复用能力。在 FedAvg 主实验、更强异构、本地训练增强和 FedProx 扩展设置下，插件结果分别比原方法低 3.39、5.22、1.34 和 1.63 个百分点。该结果说明，FedFed 机制被封装为即插即用插件后，仍能保留主要性能收益，并使特征蒸馏模块能够独立接入不同联邦主流程。

机制诊断和消融实验验证了插件关键模块的必要性。敏感特征具有较强分类能力，鲁棒特征分类能力较弱，说明特征蒸馏过程能够将分类相关信息集中到共享特征中。关闭特征蒸馏阶段、关闭共享特征主训练或关闭蒸馏增强策略后，模型性能均明显下降，表明插件性能来自多个模块的协同作用。

跨优化器实验表明，FedFed 插件具备跨联邦优化器接入能力。插件在 FedProx、FedAvgM 和 FedNova 上带来明显提升，在 SCAFFOLD 上提升较小。SCAFFOLD 通过控制变量校正客户端梯度方向，FedFed 插件通过共享特征改变本地训练信号，二者同时作用于客户端更新过程，因此插件增益会受到底层优化机制影响。

隐私验证表明，范数剪裁与高斯噪声能够降低共享特征的模型反演风险。最佳



配置下，分类代理准确率仅小幅下降，模型反演 PSNR 由 18.87 降至 16.58，说明共享特征中的可恢复图像信息减少。但成员推断实验显示，该机制不能稳定降低 MIA 风险，因此本文方法不属于严格隐私保护机制，仍需结合动态噪声、正则化训练或差分隐私方法进一步改进。

总体来看，本文完成了 FedFed 特征蒸馏思想从算法机制到插件化系统的实现，并通过性能实验、原方法对照、机制诊断、消融实验、隐私验证、跨优化器实验和开销分析验证了其有效性与局限性。后续工作可从三个方向展开：一是优化特征蒸馏与共享池维护策略，降低插件训练开销；二是设计更稳定的共享特征隐私保护机制；三是扩展到更多数据集、模型结构和真实联邦部署场景，进一步验证插件化特征蒸馏的泛化能力。

## 参考文献

- [1] Kairouz P, McMahan H B, Avent B, et al. Advances and open problems in federated learning[J]. Foundations and Trends in Machine Learning, 2021, 14(1-2): 1-210.
- [2] Yang Q, Liu Y, Chen T, et al. Federated machine learning: Concept and applications[J]. ACM Transactions on Intelligent Systems and Technology, 2019, 10(2): 1-19.
- [3] McMahan B, Moore E, Ramage D, et al. Communication-efficient learning of deep networks from decentralized data[C]//Proceedings of the 20th International Conference on Artificial Intelligence and Statistics. PMLR, 2017: 1273-1282.
- [4] Li Q, Diao Y, Chen Q, et al. Federated learning on non-IID data silos: An experimental study[C]//Proceedings of the 38th IEEE International Conference on Data Engineering. IEEE, 2022: 965-978.
- [5] Li T, Sahu A K, Zaheer M, et al. Federated optimization in heterogeneous networks[C]//Proceedings of Machine Learning and Systems. 2020, 2: 429-450.
- [6] Karimireddy S P, Kale S, Mohri M, et al. SCAFFOLD: Stochastic controlled averaging for federated learning[C]//Proceedings of the 37th International Conference on Machine Learning. PMLR, 2020: 5132-5143.
- [7] Wang J, Liu Q, Liang H, et al. Tackling the objective inconsistency problem in heterogeneous federated optimization[C]//Advances in Neural Information Processing Systems. 2020, 33: 7611-7623.
- [8] Reddi S J, Charles Z, Zaheer M, et al. Adaptive federated optimization[C]//Proceedings of the International Conference on Learning Representations. 2021.
- [9] Li X, Jiang M, Zhang X, et al. FedBN: Federated learning on non-IID features via local batch normalization[C]//Proceedings of the International Conference on Learning Representations. 2021.
- [10] Li Q, He B, Song D. Model-contrastive federated learning[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2021:

10713-10722.

[11] Tan Y, Long G, Liu L, et al. FedProto: Federated prototype learning across heterogeneous clients[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2022, 36(8): 8432-8440.

[12] Yang Z, Zhang Y, Zheng Y, et al. FedFed: Feature distillation against data heterogeneity in federated learning[C]//Advances in Neural Information Processing Systems. 2023, 36.

[13] Hsu T M H, Qi H, Brown M. Measuring the effects of non-identical data distribution for federated visual classification[EB/OL]. arXiv:1909.06335, 2019.

[14] Nasr M, Shokri R, Houmansadr A. Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning[C]//Proceedings of the IEEE Symposium on Security and Privacy. IEEE, 2019: 739-753.

[15] Shokri R, Stronati M, Song C, et al. Membership inference attacks against machine learning models[C]//Proceedings of the IEEE Symposium on Security and Privacy. IEEE, 2017: 3-18.

[16] Kingma D P, Welling M. Auto-encoding variational Bayes[C]//Proceedings of the International Conference on Learning Representations. 2014.

[17] Higgins I, Matthey L, Pal A, et al. beta-VAE: Learning basic visual concepts with a constrained variational framework[C]//Proceedings of the International Conference on Learning Representations. 2017.

[18] Zhang H, Cisse M, Dauphin Y N, et al. mixup: Beyond empirical risk minimization[C]//Proceedings of the International Conference on Learning Representations. 2018.

[19] Bochkovskiy A, Wang C Y, Liao H Y M. YOLOv4: Optimal speed and accuracy of object detection[EB/OL]. arXiv:2004.10934, 2020.

## 致 谢

衷心感谢导师×××教授对本人的精心指导。他的言传身教将使我终生受益。  
感谢×××教授，以及实验室全体老师和同窗们的热情帮助和支持！